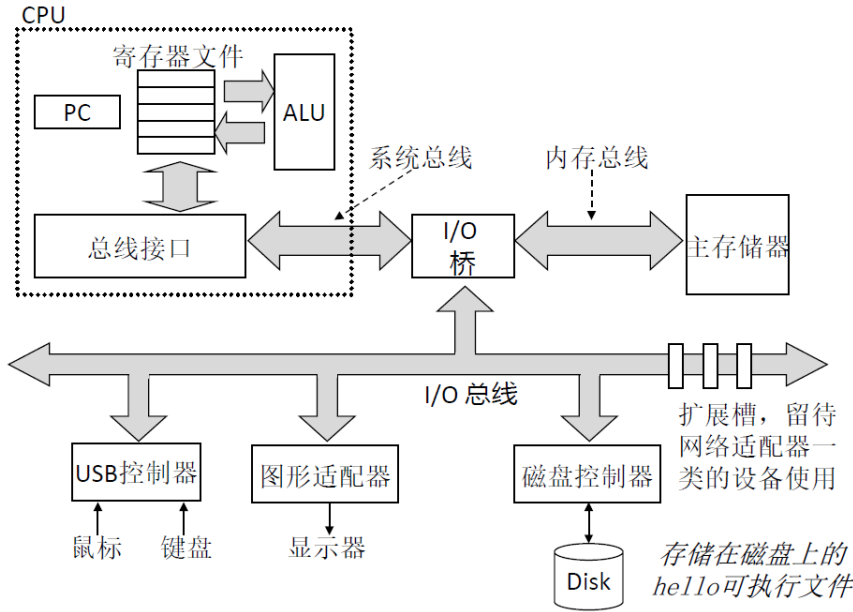
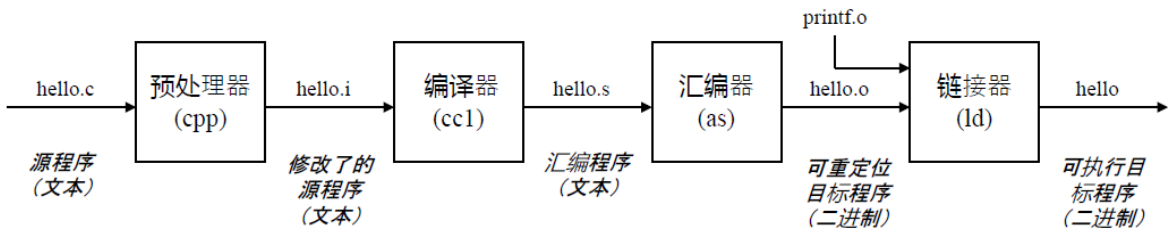


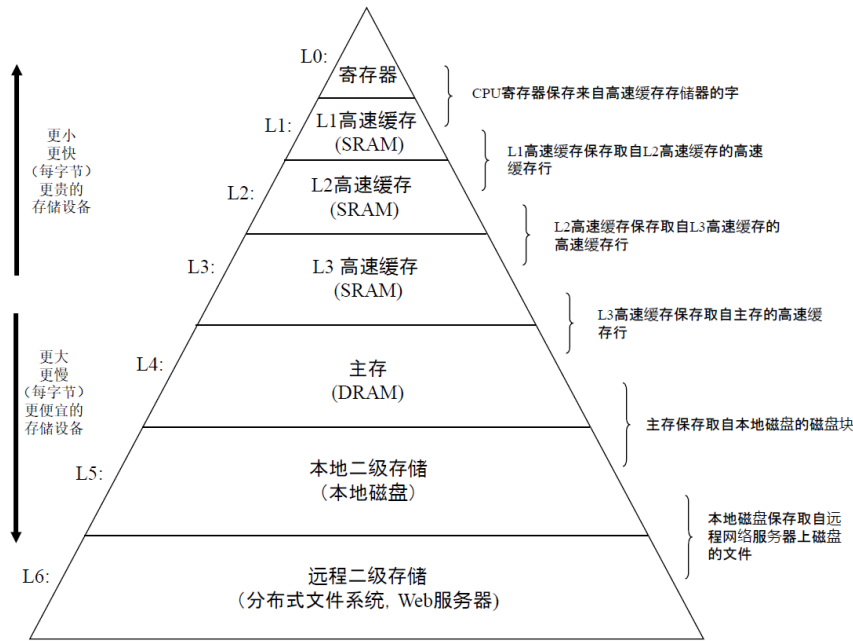
1. 编译系统:

c (文本) → i (预处理的文本) → s (汇编) → o (可重定位目标程序) → 可执行程序

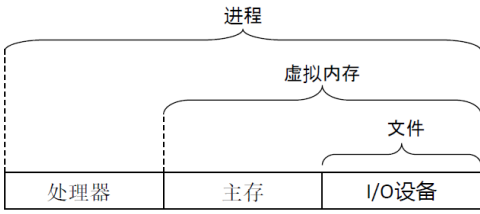


2.

3. 寄存器 → Cache (SRAM) → 主存 (DRAM) → 本地磁盘 → 远程存储

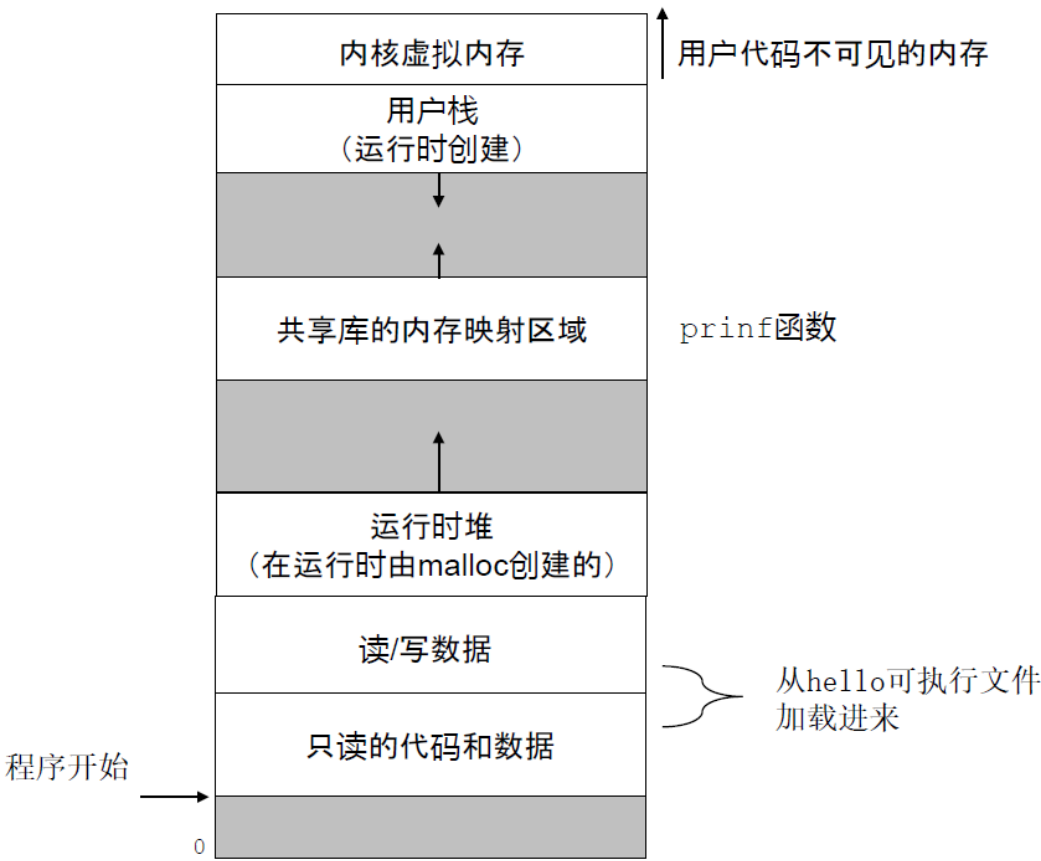


4.



5. x86-64 中，heap 需要调用时动态分配

stack 实际为 8MB, 虚拟空间为 128MB; %rsp: 栈顶指针, %rbp: 栈帧指针 (可作别用)



6. Amdahl 定律
$$S = \frac{1}{(1 - \alpha) + \alpha/k}$$

7. 响应时间：完成一件任务所需的时间；吞吐量：单位时间完成的工作量

8. CPU 时间=时钟周期数 × 时钟周期=时钟周期数/时钟频率

=指令数 (IC) × 每条指令时间周期数 (CPI) × 时钟周期

=指令数 (IC) × 每条指令时间周期数 (CPI) / 时钟频率

9. $1 + \sum_{i=0}^{w-1} 2^i = 2^w$

10.

数据类型	32 位	64 位
char	1	1
short	2	2
int	4	4
long	4	8
float	4	4
double	8	8
pointer	4	8

11. 64 位系统的地址只有 48 位

12. x 的最低有效字节，其它位都设为 0: $x \& 0xFF$

x 的最低有效字节不变，所有其它位变反（取补）: $x \wedge \sim 0xFF$

x 的最低有效字节设置为全 1，所有其它字节保持不变: $x | 0xFF$

$$A \wedge 1 = \sim A \quad A \wedge 0 = A$$

13. 区别逻辑运算符（&&、||、!）和位运算符（&、|、~）

14. 移位运算符:

<<: 右边添 0

>>: 逻辑移位左边添 0; 算数移位左边添符号位

15. $x \gg 5$ 不会改变 x 的值

16. w 位有符号数的表示

	原码	反码	补码	移码
范围	$-2^{w-1}+1 \sim 2^{w-1}-1$	$-2^{w-1}+1 \sim 2^{w-1}-1$	$-2^{w-1} \sim 2^{w-1}-1$	$-2^{w-1} \sim 2^{w-1}-1$
符号位	0 正 1 负	0 正 1 负	0 正 1 负	1 正 0 负
正负数表示	符号位+数值位	正数: 原码 负数: 数值位取反	正数: 原码 负数: 反码+1	即符号位取反后的补码
有无 0 ⁺ 0 ⁻	有	有	无	无
备注	符号位不参与运算	1. 符号位参与运算 2. 相当于 $-(8-1)421$ 码	1. 符号位参与运算 2. 相当于 -8421 码 3. 所有位取反+1 为其相反数(需考虑溢出)	无符号真值 +偏置值 =移码

17. 常量默认有符号整数，U 后缀表示无符号

18. 将 U 赋值给 S, S 会解读为有符号数; 将 S 赋值给 U, U 会解读为无符号数

19. U 和 S 都参与的运算, 会将 S 解读为 U

20. 扩展: 无符号数: 扩展位添 0; 有符号数: 扩展位添符号位

21. 无符号数加法 (补码): 相加后丢弃溢出位

22. 有符号数加法: 相加后丢弃溢出位, 考虑正/负溢

23. 对于有/无符号数来说, 左移 k 位相当于乘 2^k

对于有/无符号数来说, 右移 k 位相当于向下整除 2^k

24. 修正向上整除 2^k : $(x+2^k-1)/2^k$ 相当于给向下整除的结果+1

25. $\because x + \sim x = -1 \rightarrow \sim x + 1 = -x$

26. 大端法 (人类书写): 低位放高地址 SUM; 小端法 (硬件设计): 低位放低地址 x86, IA32

27. $\text{unsign} > -1$, 编译器会将 -1 解释为 1111...

28. 对于 $\text{int } x$, 若 $x \geq 0 \rightarrow -x \leq 0$ 成立; 若 $-x \leq 0 \rightarrow x \geq 0$ 不成立 ($x=1000\ldots$ 会溢出)

29. 浮点数: $(-1)^s M 2^E$ 真值: $(-1)^s \times 1.\text{frac} \times 2^{\text{exp}-\text{bias}}$



单精度: 1 位 8 位 23 位

双精度: 1 位 11 位 52 位

1) exp 非全 0 或非全 1:

$E = \text{exp} - \text{bias}$ (bias 为 $2^{\text{exp 的位数}} - 1 - 1$, 单精度 127, 双精度 1023)

$M = 1.\text{frac}$ 规格化

2) exp 为 0000... (存在 0^+ 和 0^-):

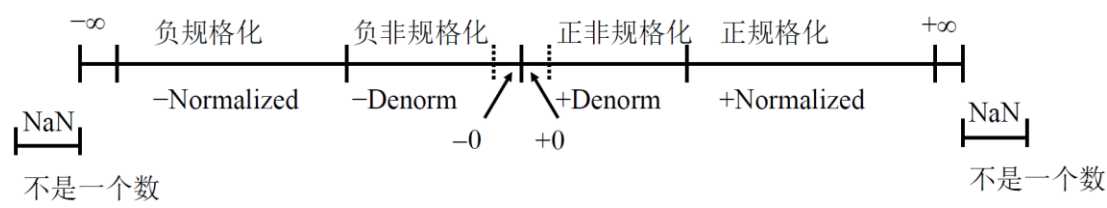
$E = 1 - \text{bias}$

$M = 0.\text{frac}$ 非规格化

3) exp 为 1111...:

① $\text{frac} = 0000\ldots$: $s=0$ 为 $+\infty$; $s=1$ 为 $-\infty$ ② $\text{frac} \neq 0000\ldots$: NaN

30.



31. 默认向偶数舍入（二进制最低为为 0）

1. XXXXGRS S 为 R 后所有位，只要有 1，S=1，只有全为 0 才有 S=0

向上舍入条件：①R=S=1 ②G=R=1, S=0

32. 浮点加/乘法结合律不成立，乘法对加法不满足分配律

33. 浮点乘法：s 异或、M 相乘、E 相加

34. double/float → int：截断 M(向零舍入)

int → double：精确转化

int → float：舍入

35. 汇编语言数据类型：

整数：1，2，4，8byte

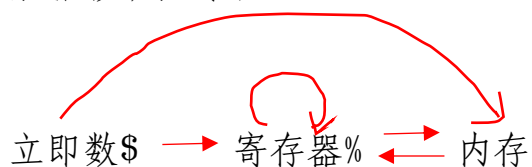
浮点数：4，8，10byte

SIMD 向量：8，16，32，64byte

36. X86-64 的整数寄存器

%rax	%eax	%r8	%r8d
%rbx	%ebx	%r9	%r9d
%rcx	%ecx	%r10	%r10d
%rdx	%edx	%r11	%r11d
%rsi	%esi	%r12	%r12d
%rdi	%edi	%r13	%r13d
%rsp	%esp	%r14	%r14d
%rbp	%ebp	%r15	%r15d

37. 数据移动关系图:



32 位有符号数

38. $D(\%a, \%b, S)$, 取 $\%a$ 中储存的地址 + $\%b$ 中储存的地址 $\times S + d$ 后形成的新地址中的值

即地址为: $a + b \times S + D$ S 一般为 1, 2, 4, 8

39. $\%a$ 访问 a 中存储的值, $(\%a)$ 访问 a 中储存地址的内存

40. `mov %a, %b` 将 a 的内容 (立即数、寄存器/内存地址中存储的值) 复制到 b

41. 简单传送指令

<code>movb</code>	半字: 1byte	不修改目标寄存器
<code>movw</code>	单字: 2byte	不修改目标寄存器
<code>movl</code>	双字: 4byte	将目标寄存器的最高 4byte 改为 0 (可减少后续操作数)
<code>movq</code>	四字: 8byte	不修改目标寄存器

42. 零扩展传送指令: `movzxy`

$x < y$: $x \rightarrow y$, 将高位清零 没有 `movzlb`、`movzlw`、`movzlwq`

43. 符号扩展传送指令: `movsxy`

$x < y$: 将 x 高位符号位 (即最高位) 扩展后, $x \rightarrow y$

44. `leaq %a, %b` 将 a 的内存地址表达式 (只能是内存地址表达式, 不能是立即数/寄存器) 的计算结果放到 b (不访存, 区别于 `mov`)

仅一条指令就能完成求和+移动, 有时候也会当乘法来使用

常用作数组的计算 `lea (%rbx,%rdi,8), %rax` `rax = &rbx[rdi]`

以及复杂的计算 `lea (%rbx,%rbx,2), %rax` `rax = rbx * 3`

45. AT&T (Unix/Linux) 最后一个参数为目标; Intel (Windows) 第一个参数为目标

AT&T: `XXX src dest` 即 `dest=dest op src`

Inet1: XXX src dest 即 dest=dest op src

46. sar (Shift Arithmetic Right) 左侧补符号位 shr (Shift Right): 左侧补 0

addq	Src, Dest	Dest = Dest + Src
subq	Src, Dest	Dest = Dest - Src
imulq	Src, Dest	Dest = Dest * Src
salq	Src, Dest	Dest = Dest << Src
sarq	Src, Dest	Dest = Dest >> Src
shrq	Src, Dest	Dest = Dest >> Src
xorq	Src, Dest	Dest = Dest ^ Src
andq	Src, Dest	Dest = Dest & Src
orq	Src, Dest	Dest = Dest Src

47.

incq	Dest	Dest = Dest + 1
decq	Dest	Dest = Dest - 1
negq	Dest	Dest = - Dest
notq	Dest	Dest = ~Dest

48. 整数参数（大于 6 个的输入参数存放在栈中）

第一个参数	%rdi
第二个参数	%rsi
第三个参数	%rdx
第四个参数	%rcx
第五个参数	%r8
第六个参数	%r9
返回值	%rax
栈顶指针	%rsp
栈帧指针（也可作别用）	%rbp
被调用者保存寄存器 （用于压退栈时保存上下文）	%rbx

49. 浮点参数: %xmm0 ~ %xmm7

返回值: %xmm0

50. 文本.c—编译器→文本.s—汇编器→二进制.o—链接器+静态库.a→p

51. x86 复杂指令集; ARM 精简指令集

52. 指令的后缀通常可以省略, 编译器的汇编输出带后缀, 反汇编不带

53. 比较指令: `cmpq a b` 会计算 $b-a$, 仅会改变标志位, 不存储结果

54. 测试指令: `testq a b` 会计算 $b\&a$, 仅会改变标志位, 不存储结果

55. 跳转指令 (本质是比较标志位)

橙色为有符号数

jmp	无条件跳转	\
je	相等跳转	ZF=1
jne	不相等跳转	ZF=0
js	负数跳转	SF=1
jns	非负数跳转	SF=0
jg	大于跳转	$b-a > 0: (SF==0F) \&\&ZF==0$
jge	大于等于跳转	$b-a \geq 0: SF==0F$
jl	小于跳转	$b-a < 0: SF!=0F$
jle	小于等于跳转	$b-a \leq 0: (SF!=0F) ZF==1$
ja	高于跳转 (有符号数)	$b > a: CF=ZF=0$
jb	低于跳转 (无符号数)	$b < a: CF=1$

56. 条件码 (leaq 指令不改变条件码)

CF	进位标志 (针对无符号数)	0: 不进位; 1: 进位
SF	符号标志 (针对有符号数)	0: 正; 1: 负
ZF	零标志	0: 零; 1: 非零
OF	溢出	0: 不溢出; 1: 溢出

SetX	条件 Condition	描述 Description
sete	ZF	等于/零 Equal / Zero
setne	~ZF	不等/不为零 Not Equal / Not Zero
sets	SF	负数 Negative
setns	~SF	非负 Nonnegative
setg	~(SF^OF)&~ZF	大于（有符号） Greater (Signed)
setge	~(SF^OF)	大于或等于（有符号数） Greater or Equal (Signed)
setl	(SF^OF)	小于（有符号数） Less (Signed)
setle	(SF^OF) ZF	小于或等于（有符号数） Less or Equal (Signed)
seta	~CF&~ZF	高于（无符号数） Above (unsigned)
setb	CF	低于（无符号数） Below (unsigned)

57.

58. setl

SF	OF	含义
0	0	无溢出，结果非负 → 不小于
1	0	无溢出，结果为负 → 小于
0	1	负溢 → 小于
1	1	正溢 → 不小于

59. 条件跳转指令: `cmovXX src dest`

条件成立(由后缀 XX 决定)则执行 `dest=src`，否则保持 `dest` 不变

60. 指令的后缀: g: greater l: less e: equal n: not a: above b: below

61. 指令的前缀: v: vector (常出现在 SIMD)

62. 条件跳转传送的风险: ①需要计算不同分支的结果(开销大); ②原判断条件用于判断操作是否合法时，提前计算不同分支会进行不合法的操作; ③计算有副作用

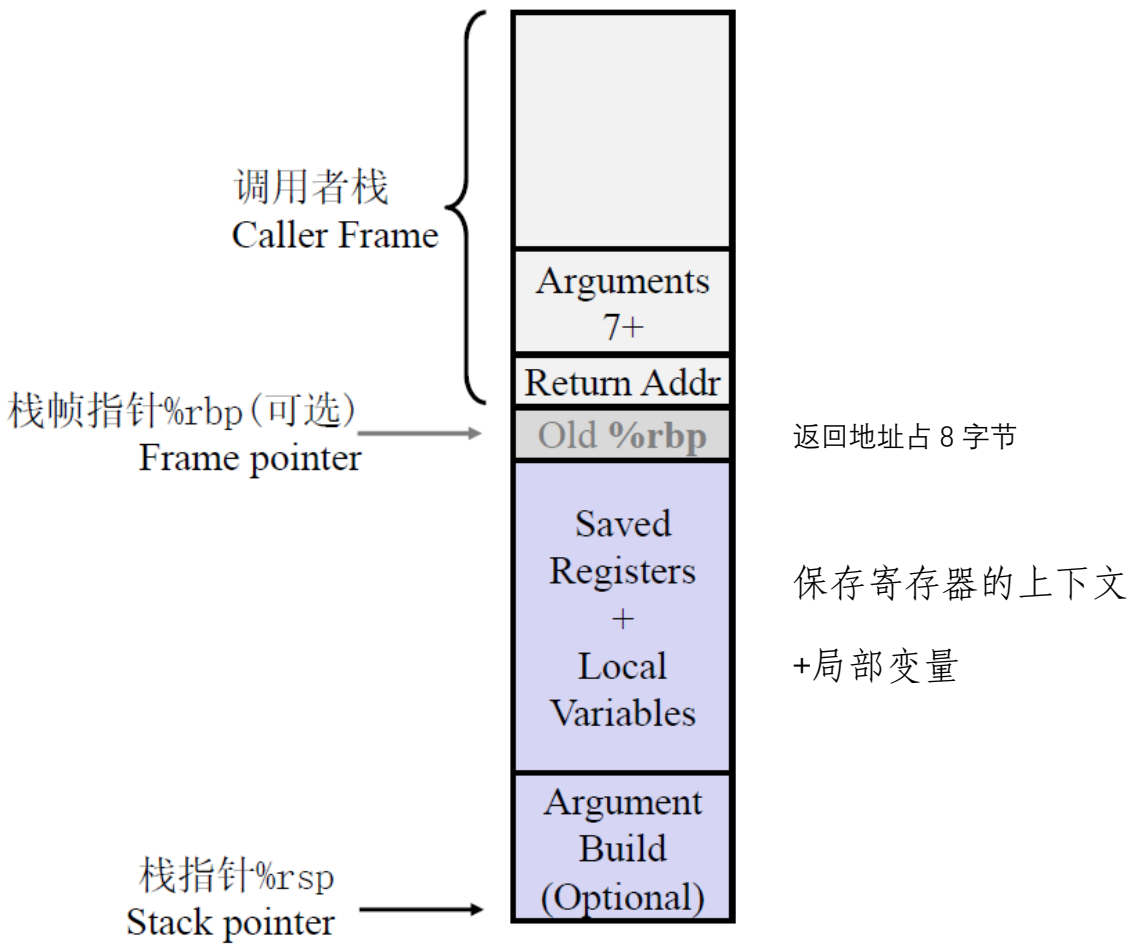
63. `setl` 和 `movzblq` 不修改条件码

64. `switch` 语句会生成 `jtab`(每行 8byte), 间接跳转时 8 作比例因子: `jmp *.L4(%rdi,8)`

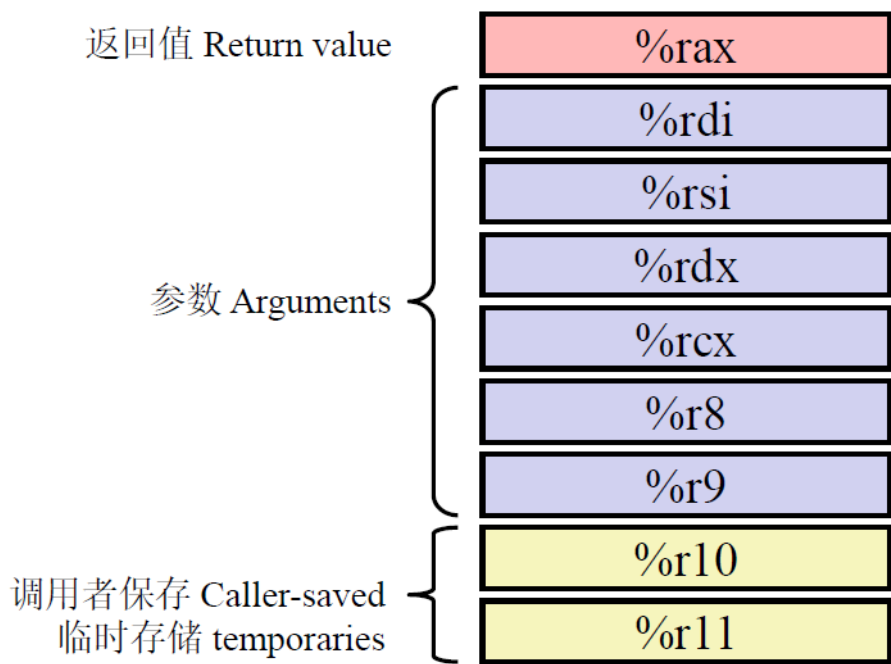
65. 栈从高地址向低地址生长, `%rsp` 记录最低栈的地址(栈顶)

`push %rsp--=8` `pop %rsp+=8`

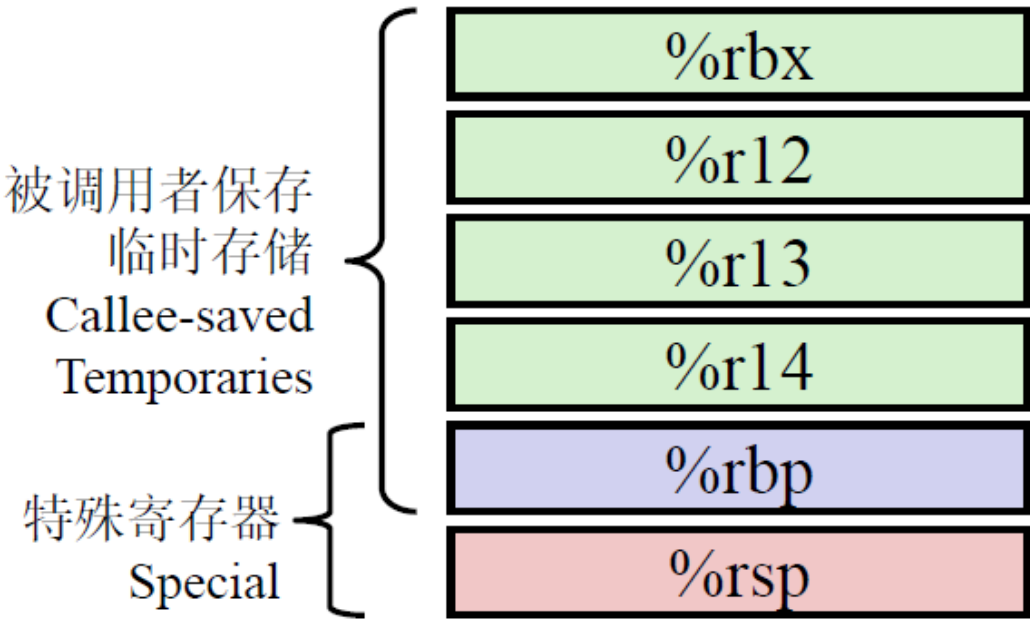
66. 函数调用都会生成新栈帧（直接调用自己也不例外），被调用者先返回
67. “一格”是 8 字节



68. 调用者保存（返回值+6 个参数寄存器+%r10,%r11）不用记

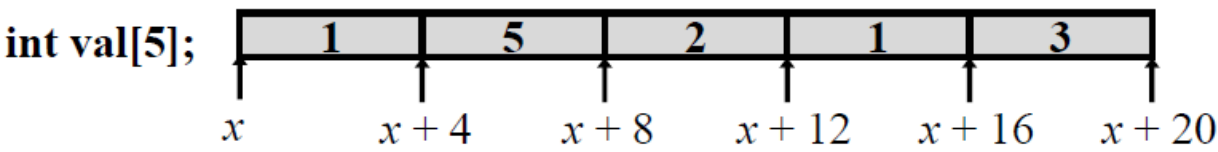


69. 被调用者保存（由被调用者恢复）不用记



70. x86-64 中，寄存器参数按顺序压入；栈参数按从右到左的顺序压入（反向压入）

71.



引用Reference	类型Type	值Value
val[4]	int	3
val	int *	x
val+1	int *	x + 4
&val[2]	int *	x + 8
val[5]	int	??
*(val+1)	int	5
val + i	int *	x + 4 i

val+n ⇔ &(val[n]) ⇔ val+n*sizeof(element)

*(val+n) ⇔ val[n]

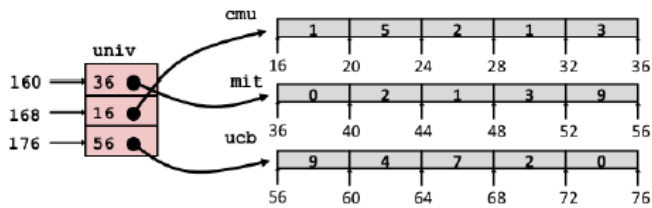
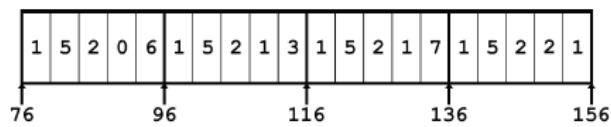
72.

名称	A	*A	**A	***A	A 指向的内容
int A[3]	3 × 4	4	\	\	
int*A	8	4	\	\	
int*A[3] int(*A)[3]	3 × 8	8	4	\	
int(*A)[3]	8	3 × 4	4	\	
int A[3][5]	3 × 5 × 4	5 × 4	4	\	
int*A[3][5] int*(A[3][5])	3 × 5 × 8	5 × 8	8	4	
int(*A)[3][5]	8	3 × 5 × 4	5 × 4	4	
int(*A[3])[5]	3 × 8	8	5 × 4	4	

73.

嵌套数组

多级数组



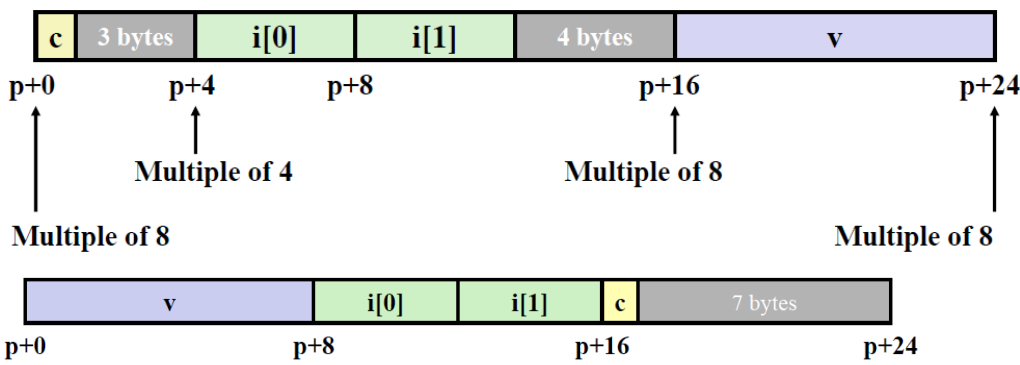
嵌套数组仅需一次访存即可

多级数组需要进行两次读存操作，第一次读取行数组指针，第二次访问数组元素

74. 结构体内部的对齐规则：每一项数据的起始地址必须是该数据大小的整数倍

同时结构体之间的对齐规则：必须是结构体内部最大元素数据大小的整数倍

显然占 2ⁿ 字节的数据类型的地址的最低 n 位必须为 0



75. 8MB 为栈实际大小，128MB 为栈的虚拟空间大小

■ 栈 Stack

- 运行时栈（8MB限制）Runtime stack (8MB limit)
- 例如局部变量 E. g., local variables

■ 堆 Heap

- 需要时动态分配 Dynamically allocated as needed
- 当调用函数时：When call malloc(), calloc(), new()

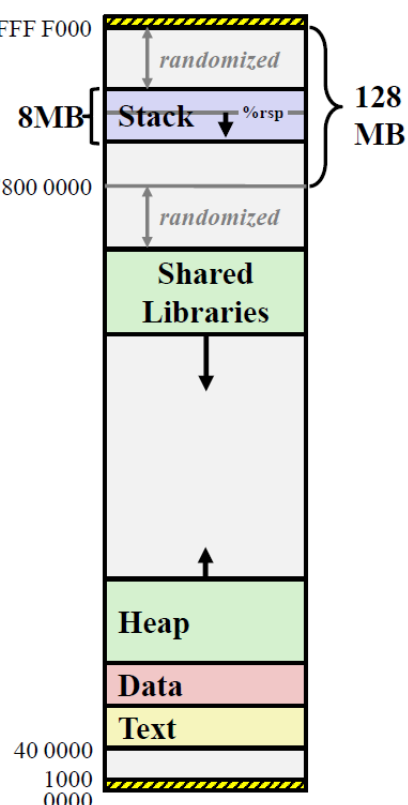
■ 数据 Data

- 静态分配的数据 Statically allocated data
- 例如全局变量、静态变量、串常量
 - E.g., global vars, static vars, string constants

■ 文本 / 共享库 Text / Shared Libraries

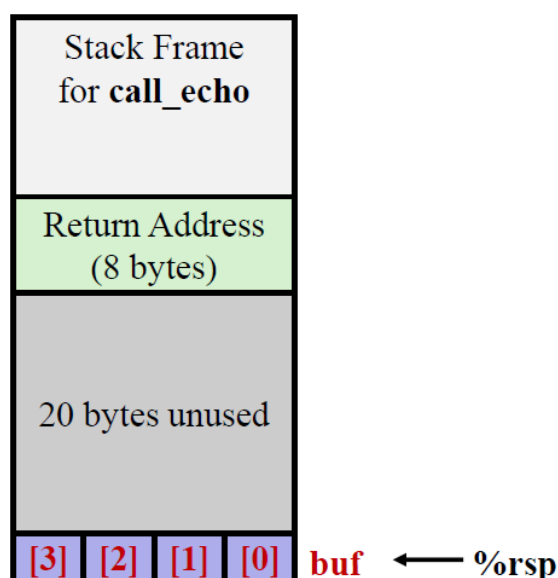
- 可执行机器指令 Executable machine instructions
- 只读的 Read-only

十六进制地址 Hex Address



76. 通过函数调用的首句 `sub $a, %rsp` 可得知栈帧大小为 `0x a`

77. 缓冲区溢出攻击：传入的数据不做检查覆盖 buf 和填充区后，覆盖了返回地址



gets, strcpy, strcat, scanf, fscanf, sscanf

78. 避免溢出漏洞：① fgets, strncpy 代替 gets, strcpy ② 栈偏移随机化 (栈帧大小随机) ③ 栈区域标记为非可执行代码段 ④ 栈金丝雀：原栈帧填充区中填入某值，退出前检查该值是否被破坏 ⑤ 用 malloc 分配内存 (注入数据存储在堆中，影响不到栈)

- 79. 小工具构造：从程序或库中寻找以 ret 结尾的代码进行构造（无法绕过金丝雀）
- 80. 联合体: 将同一区域的数据按照不同的方式解读; 强制转化: 将数据的值进行改变
- 81. 编译器优化不改变复杂度，仅改变常量因子，且一般不会优化有副作用的代码
- 82. 常见编译器优化措施 局部优化 全局优化

措施	操作
常量折叠	用常量代替已知结果的运算
强度削弱	一般用位移和加法以及内存访问表达式代替乘除法
死代码删除	删除不会被执行或者立马被覆盖的代码
公共子表达式共享	定义一个变量来表示某个多次出现的表达式
内联函数	将多个函数整合在一起，减少函数调用，同时为其他优化创造条件
代码外提	将循环内结果不变的运算提出循环
循环转换	① 按高速缓存友好顺序迭代循环(避免按列访问,提高 cache 命中率) ② 合并具有相同限制的相邻循环(一般可合并的两个循环中一个对另一个的结果是只读的) ③ 循环展开：将原迭代步长增大（但总迭代次数需为步长的整数倍，且循环内的逻辑也应当改变）常与重结合计算联系
消除别名 (内存别名: 同一块内存地址被不同名称的变量指向，导致编译器不会对此进行优化)	① 使用局部变量代替某一别名 ② 使用 restrict 告诉编译器非别名 注意：消除别名前后结果可能不一致
指针代替数组索引/函数	指针代替数组索引（包含乘法计算，有隐藏开销）/函数（函数设计中可能含有边界检查，减少调用函数导致的边界检查）
重结合计算	$((1+2)+3)+4 \rightarrow (1+2)+(3+4)$ 用多个寄存器暂存中间结果

83.

	编译型语言	解释型语言
代表	C/C++	Python
处理流程	源代码 → 一次性编译 → 机器码 → 执行	源代码 → 逐行解释执行
执行载体	二进制可执行文件	解释器实时翻译
执行速度	快(直接运行机器码)	慢(需要逐行翻译)
跨平台性	差(需为不同系统编译)	好(解释器屏蔽系统差异)

84. 机器相关优化:

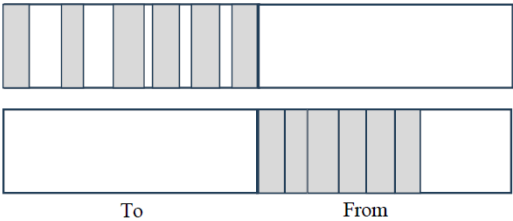
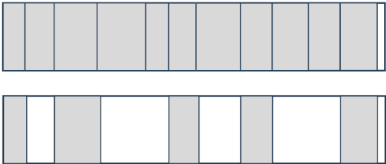
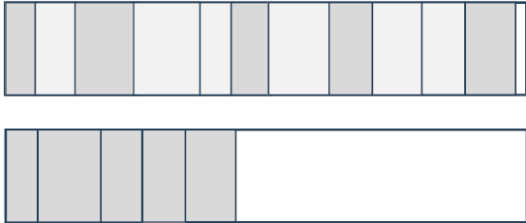
- ① 超标量处理器：一个周期发出和执行多条指令 指令级并行
- ② 分支预测：在判断分支条件的同时预执行下步操作：
 - 1. 选择分支：预执行分支跳转后的指令
 - 2. 不选择分支：预执行顺序的下一条指令(不预执行跳转后的指令)

简单启发：对循环选择分支；对 if 语句不选择分支

分支预测优化：目的减少分支数

 - 1. 合并循环
 - 2. 循环展开：常与重结合计算联系
 - 3. 先进行修改操作，若不满足条件再使用条件选择指令 comv

回溯
- ③ 流水线(除法无流水)
- ④ 调度控制：重新安排指令顺序，保持 CPU 满负荷运转
- ⑤ 向量化
- ⑥ 循环展开+重结合计算：边际效益递减(并行度越高使用寄存器越多，有开销)

方法	操作过程	弊端
存活对象标记算法	堆栈和寄存器中存活的对象即根对象，从根对象出发按照引用关系能够抵达的即为存活对象	对象标记阶段需停止应用程序的运行
引用计数算法	每个对象设置一个记录被引用从次数的计数器，计数器值为 0 时回收	① 计数器的更新增加开销 ② 循环引用无法回收 ③ 联合体的歧义导致无法判断存储的是对象引用还是数值
复制算法	将堆分成 From 和 To 两个大小相同的区域，在其中一个区域分配对象，该区域满后将存活对象拷贝到另外一个区域紧密排布 收集前 From To 	① 仅一半堆空间可用 ② 频繁移动对象，造成额外开销 ③ 移动对象后，存活对象的标记也需要更新
标记清除算法	堆满后标记存活对象，死亡对象直接抛弃 收集前 	① 频繁创建删除对象造成空间碎片化 ② 移动对象后，存活对象的标记也需要更新
标记压缩算法	堆满后标记存活对象，死亡对象直接抛弃后将存活对象紧密排布 收集前 	① 频繁移动对象，造成额外开销 ② 移动对象后，存活对象的标记也需要更新
按代收集算法	新生代进一步划分为 Eden 和 S0 和 S1 三个区域→Eden 分配对象→Eden 满后标记存活对象，抛弃死亡对象后移动到 S0→Eden 分配对象→Eden 满后标记存活对象(包括 S0)，抛弃死亡对象后全部移动到 S1→S0 与 S1 之间的相互移动会增加存活对象的年龄→存活对象年龄达到阈值后提升到旧世代→旧世代满后触发主收集（标记压缩算法） eden S0 S1 Tenured Permanent 新生代: 短; 旧世代: 长; 持久代: 元数据, 自定义函数, 库	

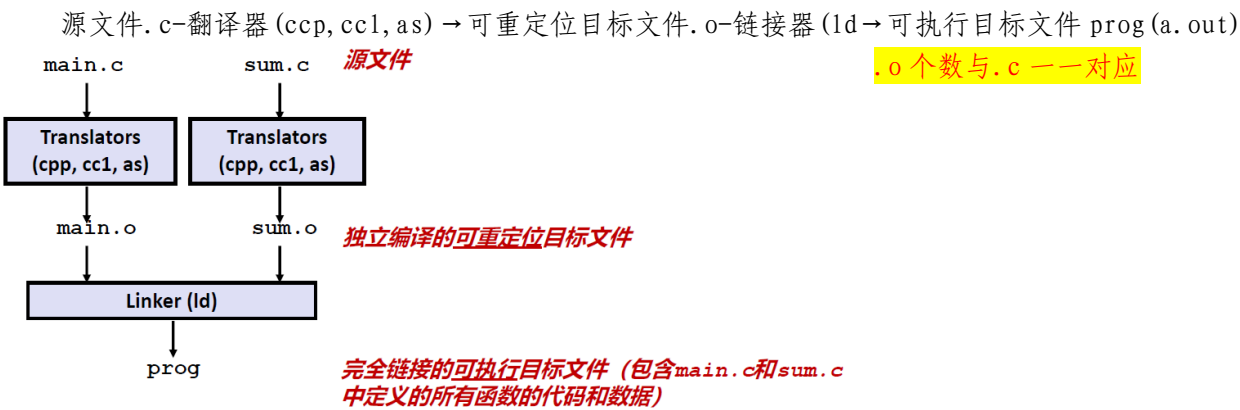
85. 垃圾收集算法的比较，见上图

86. 垃圾收集器只能自动释放堆中 new 的对象,但无法释放这些对象持有的系统资源,需要显示调用 close 来进行释放

87. 串行垃圾收集适用小规模, 并行/并发垃圾收集适用大规模

并行垃圾收集器	并发标记-清除垃圾收集器
多线程进行垃圾收集	垃圾收集器和应用程序的并发运行

88. 静态链接:



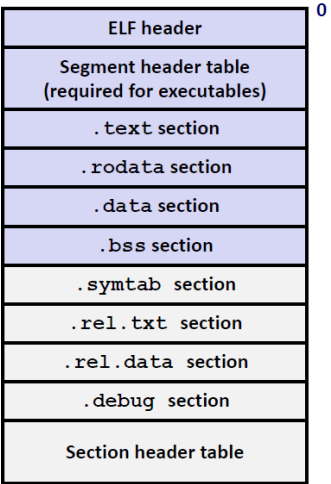
89. 链接器工作: 符号解析 (查符号表将符号引用与符号定义关联)+重定位 (将独立的代码和数据合并, 将符号在.o 中的相对位置重定位到可执行文件的绝对位置, 更新符号引用)

90. ELF (可执行可链接格式): .o a.out .so (共享目标文件, 一般加载到共享库)

	静态链接	动态链接
链接时机	编译	加载/运行
库文件形式	.a	.so
代码合并方式	库代码复制到可执行文件中	仅记录库的引用, 不复制代码
可执行文件大小	较大 (包含所有依赖库代码)	较小 (仅含自身代码和引用信息)
内存占用	每个进程独立副本 (无共享)	多进程共享同一份库代码

91. 动静态链接的比较, 见上图

92. ELF 目标文件格式



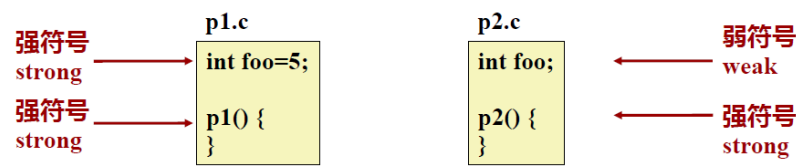
部位	作用	备注
ELF header	记录文件基本信息	必选
Segment header table	记录段页信息，描述虚实映射关系	可执行文件必选，可重定位文件（.o）可选
.text section	代码	必选
.rodata section	只读数据	必选
.data section	已初始化的全局变量	必选
.bss section	未初始化的全局变量	必选，有节头部但不占空间
.symtab section	符号表、过程和静态变量名字、节名和位置	必选
.rel.textsection	.text 段的重定位信息：标记需修正地址的指令	仅重定位文件（.o）包含
.rel.datasection	.data 段的重定位信息：标记需修正地址的数据	仅重定位文件（.o）包含
.debug section	调试信息	可选
Section header table	记录节的结构，每个节的偏移量和大小	必选

93.

符号类型	定义	举例
全局符号	本模块中定义的可被其他模块应用的符号	非静态全局变量
外部符号	其他模块定义的，被本模块应用的全局符号	
局部符号	本模块定义并使用的符号	static 限定的函数和全局变量

94. 局部非静态 C 变量存储在栈上；局部静态 C 变量存储在.bss/.data 中

95. 强符号: 有函数体的全局函数/初始化的全局变量; 弱符号: 未初始化的全局符号



- ① 不允许多个同名强符号
- ② 强/弱符号同名，选择强符号
- ③ 多个弱符号同名，任选其一

96. gcc 中

COMMON: 未初始化的全局变量，服务于外部链接的弱符号（全局变量），支持跨文件合并

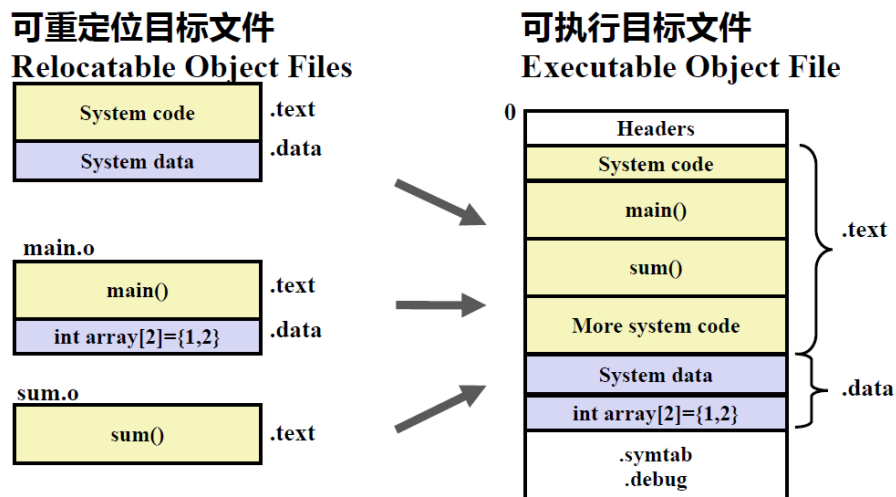
.bss: 未初始化的静态变量以及初始化为 0 的全局或静态变量(即静态变量和零初始化全局变量), 服务于内部链接的变量（静态变量、零初始化全局变量），保证作用域封闭性

97. 为什么未初始化的静态变量放到.bss 中而不是 COMMON 中？

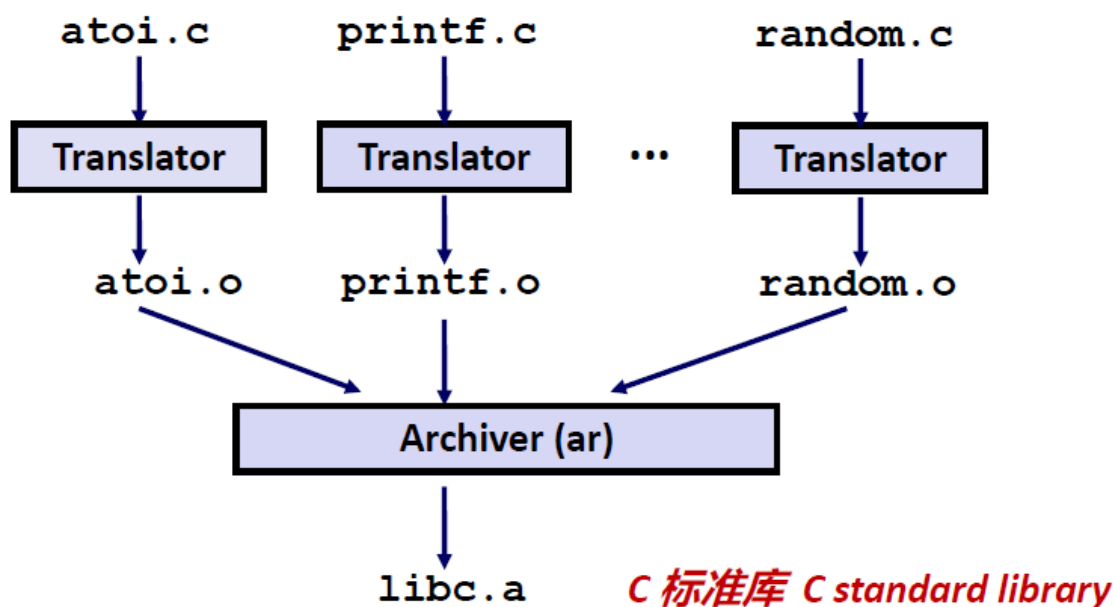
COMMON 的设计是为了解决 “跨文件的弱符号冲突”，而静态变量的内部链接属性(仅作用于当前文件)，无需跨文件

98. 头文件.h 存放符号的声明(不存放定义是避免重定义), 告诉编译器某符号存在，但定义在别处

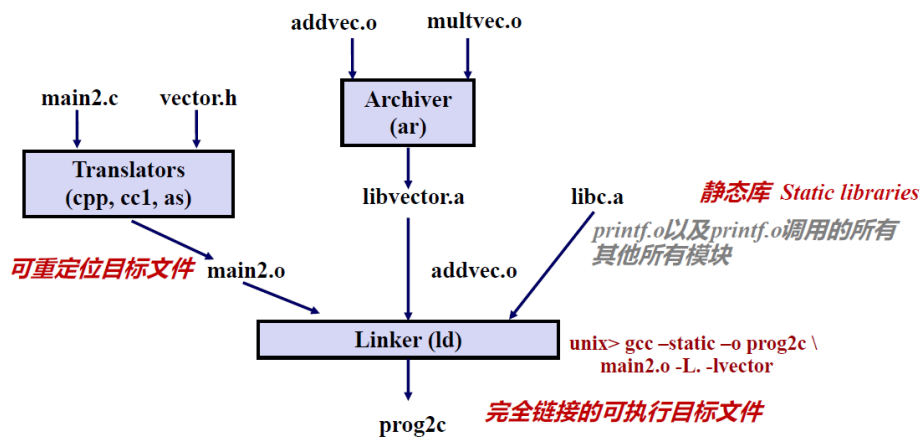
99. 重定位的图解



100. 静态库 (.a 存档文件) 的创建

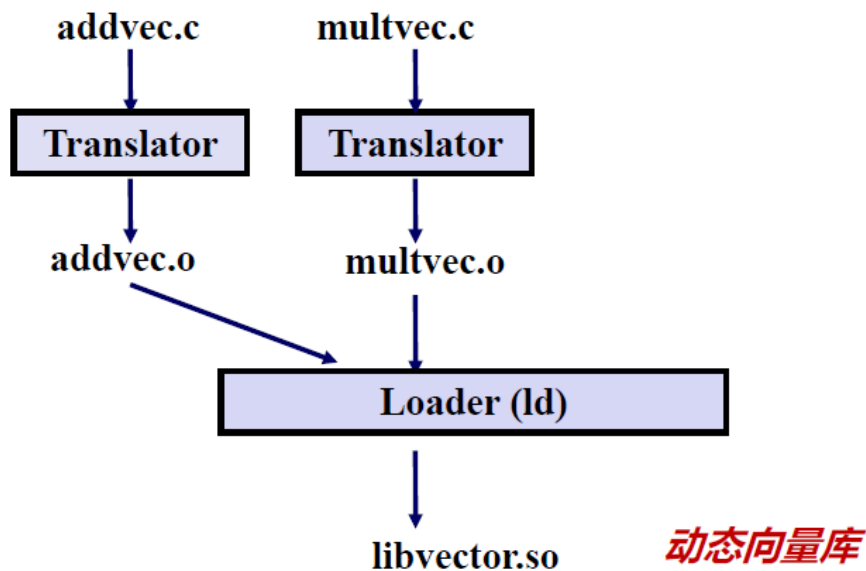


101. 与静态库的链接过程



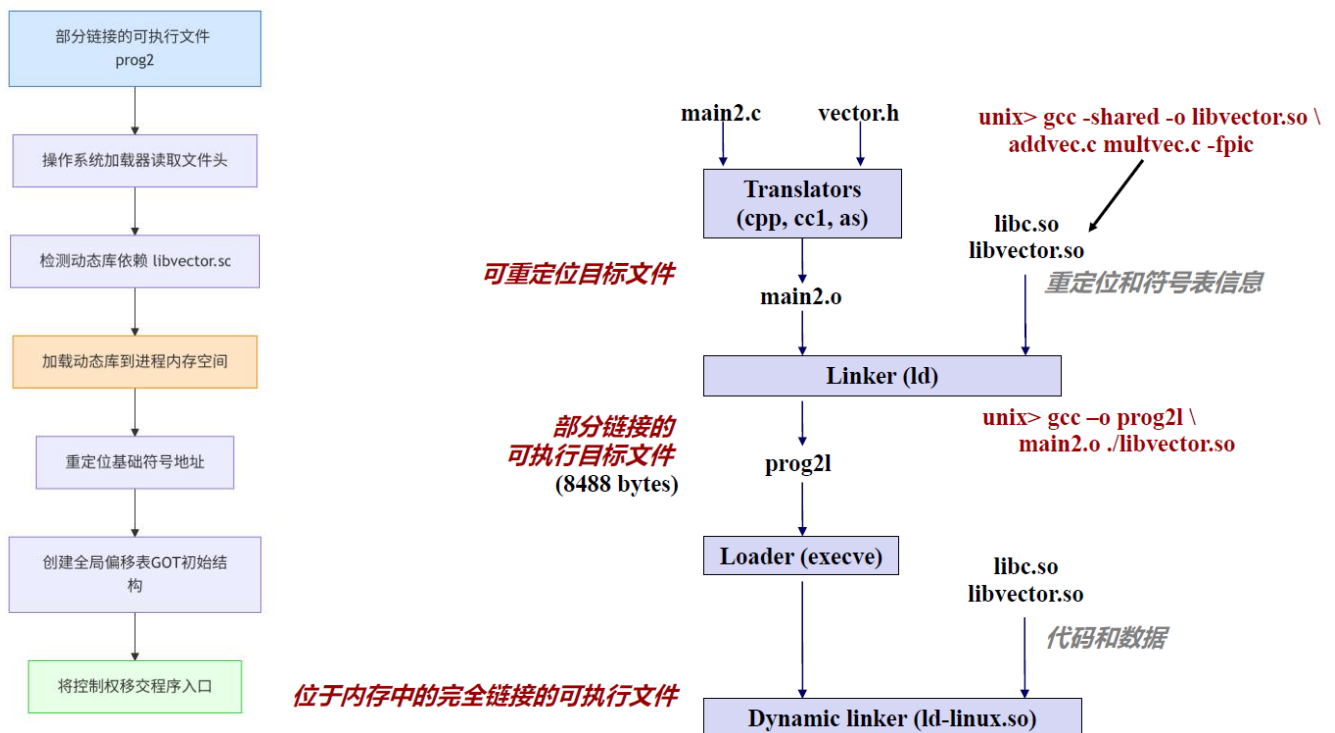
“c” 表示“编译时 (compile-time)” (861,232 bytes)

102. 按顺序扫描.o/.a文件，不回退，故一般将库函数放在结尾处
103. 动态库需要.interp section(指定使用的动态链接器)以及.dynamic section(指定使用的动态库名字)
104. 动态库(.so)的创建



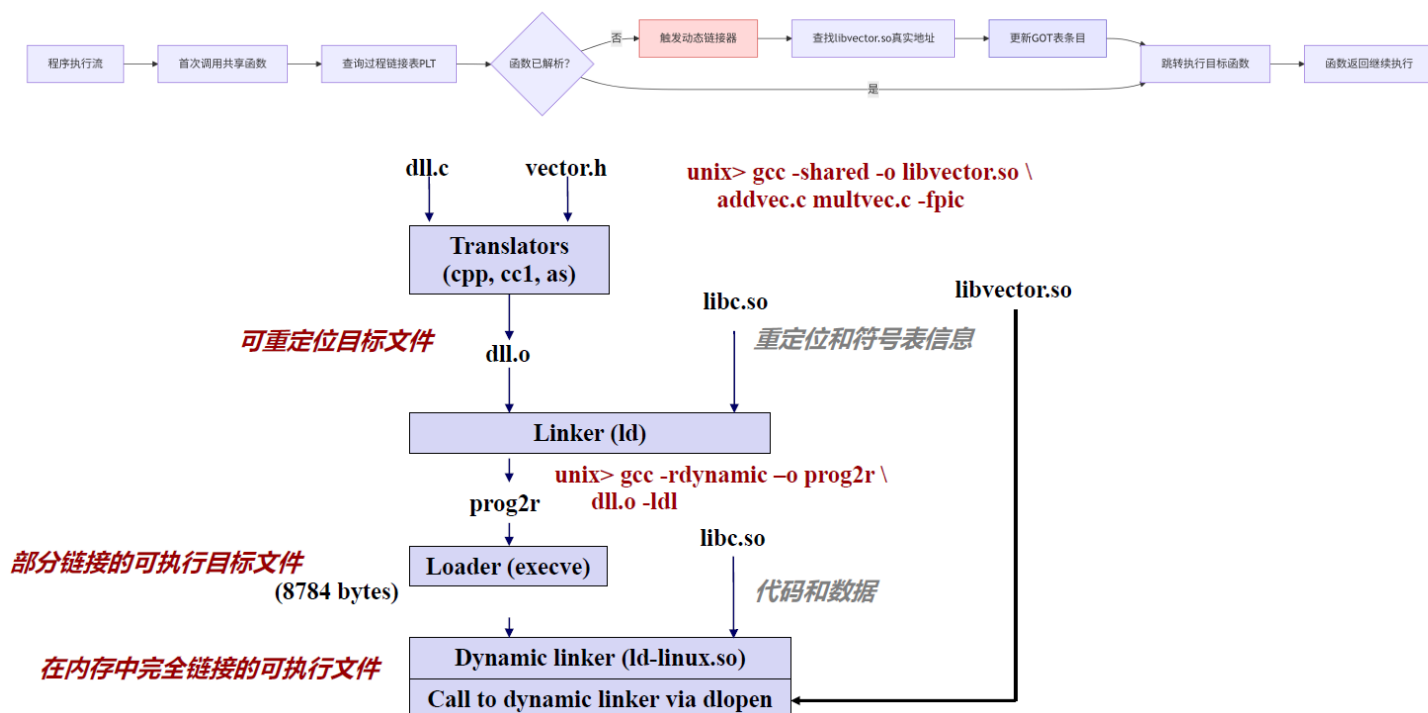
105. 加载阶段的动态链接

编译.c生成动态库.so→链接.so与.o生成部分链接的可执行文件prog2l→运行时系统加载器加载prog2l，动态链接器加载.so→生成完全链接的可执行程序



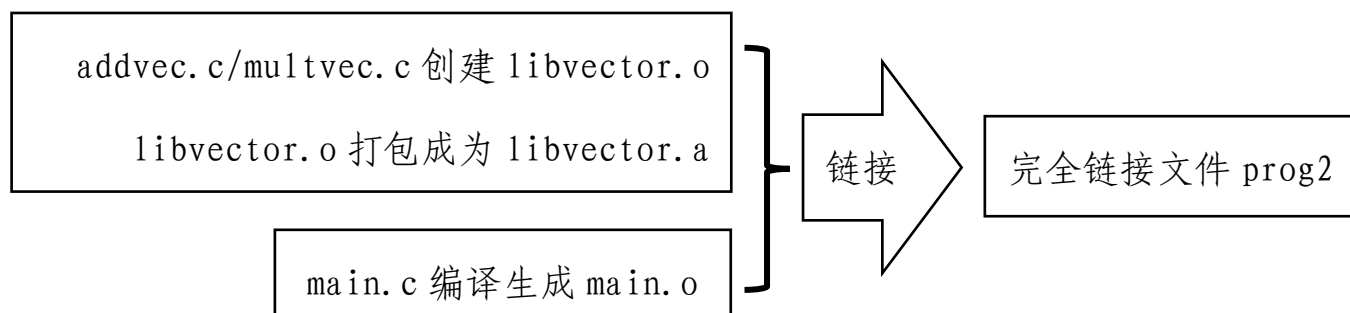
106. 运行阶段的动态链接

编译 `addvec.c`、`multvec.c` 生成动态库 `.so` → 将 `dll.c` 编译为 `dll.o` → 生成部分链接的可执行文 `prog2r` → 运行时，系统加载器加载 `prog2r` → 动态链接器 (`ld-linux.so`) 除处理默认依赖 (如 `libc.so`)，还通过程序内 `dlopen` 调用主动加载动态库 `.so` → 生成完全链接的可执行程序

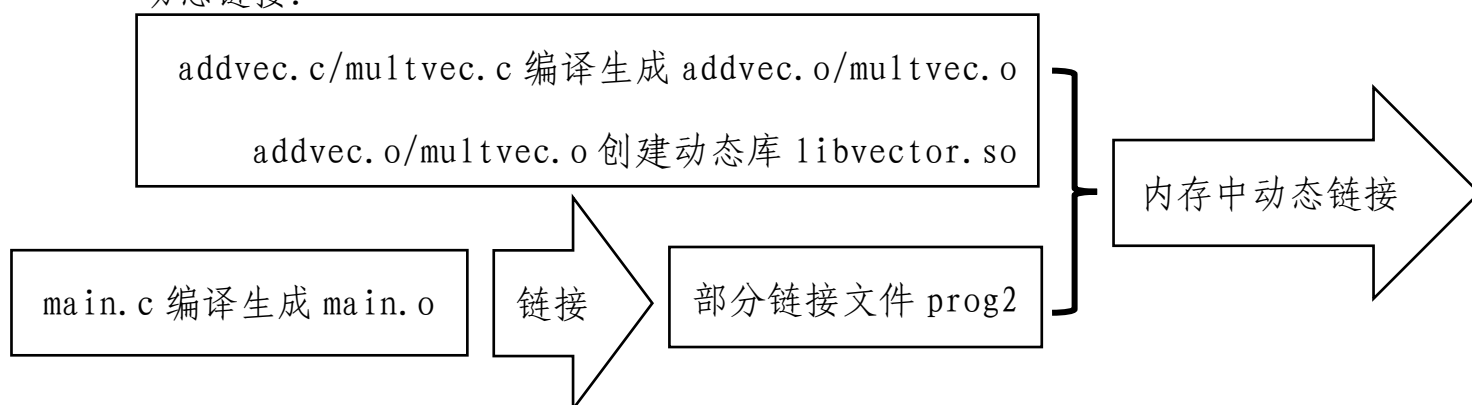


107. 动静链接区别

静态链接:



动态链接:



108. 库打桩（在不修改源码的情况下添加监控、调试或定制功能）时机：

① 编译阶段：编译源代码

#ifdef COMPILETIME

自定义包装函数 mymalloc 和 myfree，拦截 malloc 和 free，透明地跟踪程序的内存分配和释放操作，在每次分配或释放内存时打印出详细信息

② 链接阶段：可重定位目标文件被静态链接形成可执行目标文件

#ifdef LINKTIME

利用链接器（如 ld）的符号替换机制：编译源代码时，将自定义的包装函数 --wrap-malloc 和原始函数声明 __real-malloc 一同编译到目标文件中。链接阶段，通过链接器参数（如 gcc -Wl,--wrap,malloc）强制将所有调用 malloc 的地方替换为调用 __wrap-malloc，而 __real-malloc 则被替换为 malloc。

③ 加载/运行阶段：可执行目标文件被加载进内存/动态链接，并被执行时

#ifdef RUNTIME

利用操作系统动态链接器的 LD_PRELOAD 环境变量：在程序启动前，通过 LD_PRELOAD 将自定义的共享库（mymalloc.so）优先加载到进程地址空间，其符号会覆盖后续加载的标准库（如 libc.so）中的同名符号。即当动态链接器对未被解析的应用进行解析时，优先在 mymalloc.so 中进行查找。

编译	将 malloc 和 free 替换成带检测功能的自定义函数 mymalloc 和 myfree	#ifdef COMPILETIME
链接	利用链接器的特殊名字解析功能，将 malloc 替换为 __wrap-malloc，将 __real-malloc 替换为 malloc	#ifdef LINKTIME
加载/ 运行	LD_PRELOAD 环境变量告诉动态链接器对未被解析的引用（如 malloc）进行解析时，优先在 mymalloc.so 中解析查找	#ifdef RUNTIME

109. PIC (数据无关代码): 编译时生成不依赖绝对地址的代码, 通过“相对偏移”访问数据

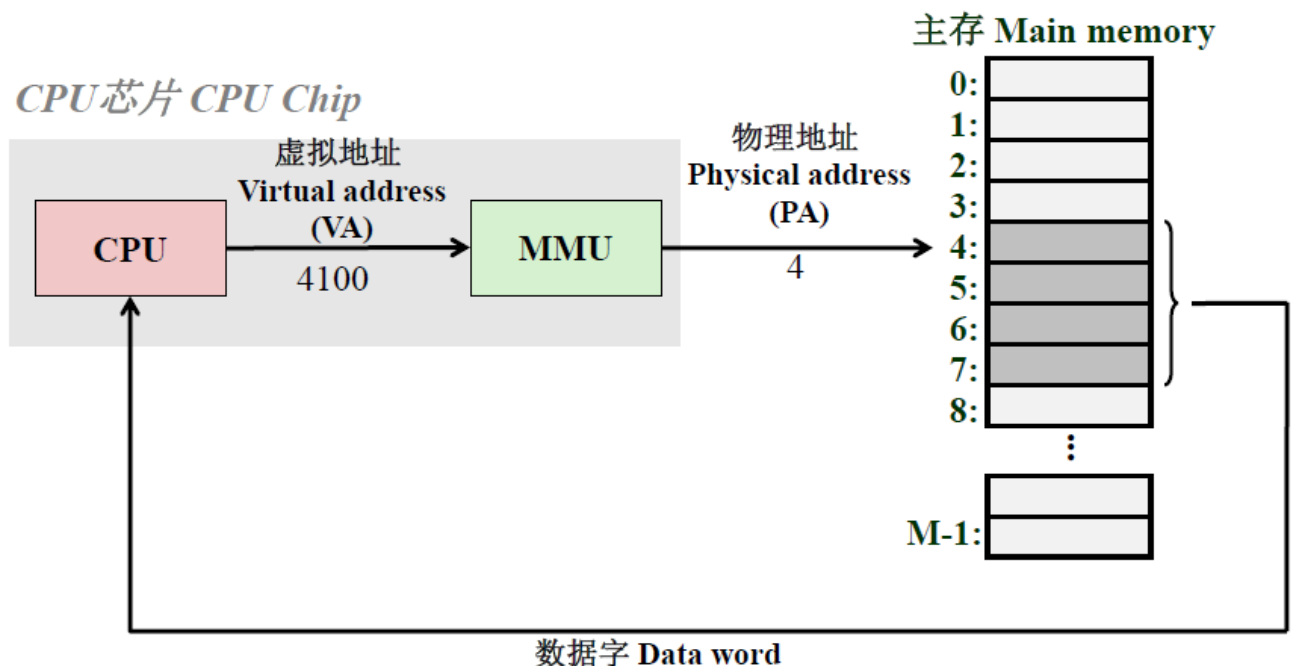
110. PIC 延迟绑定: 第一次调用时加载并解析真实地址, 将真实地址更新至 GOT, 后续调用的开销仅为一条指令和一个内存间接引用 (查找 GOT 即可得到地址)

GOT[0], GOT[1]: 存储动态链接器解析函数地址时需要的信息

GOT[2]: 存储动态链接器的入口地址

GOT[N] $N > 2$: 其余条目对应一个被调用的函数, 对应一个

111.



112. 页大小: 4KB, 采用全相连映像, 采用写回机制而不是写直达机制

113. 页表条目 PTE 记录虚拟页与物理页之间的映射关系

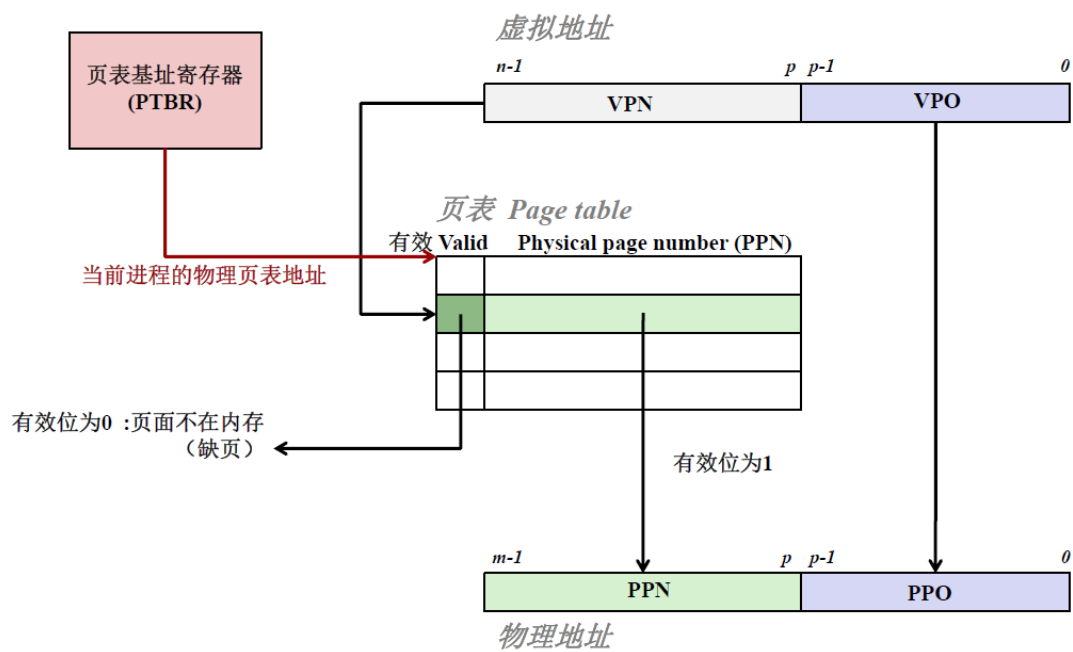
114. 页命中/中断: 引用的虚拟页在/不在物理内存中

115. 虚拟地址 $VA = \text{虚拟页号 VPN}(\text{TLBT 标记} + \text{TLBI 索引}) + \text{虚拟页偏移量 VP0}$

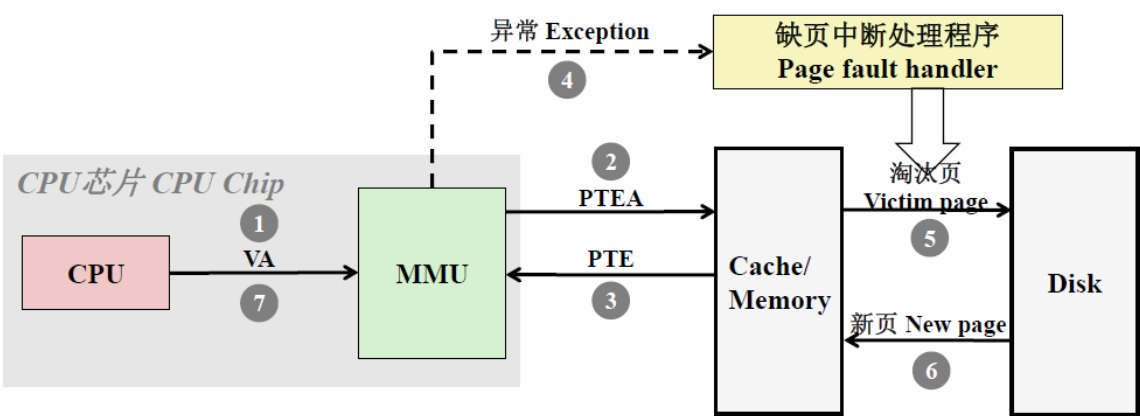
物理地址 $PA = \text{物理页号 PPN} + \text{物理页偏移量 PP0}$

$= \text{Cache 标记 CT} + \text{Cache 索引 CI} + \text{Cache 偏移量 C0}$

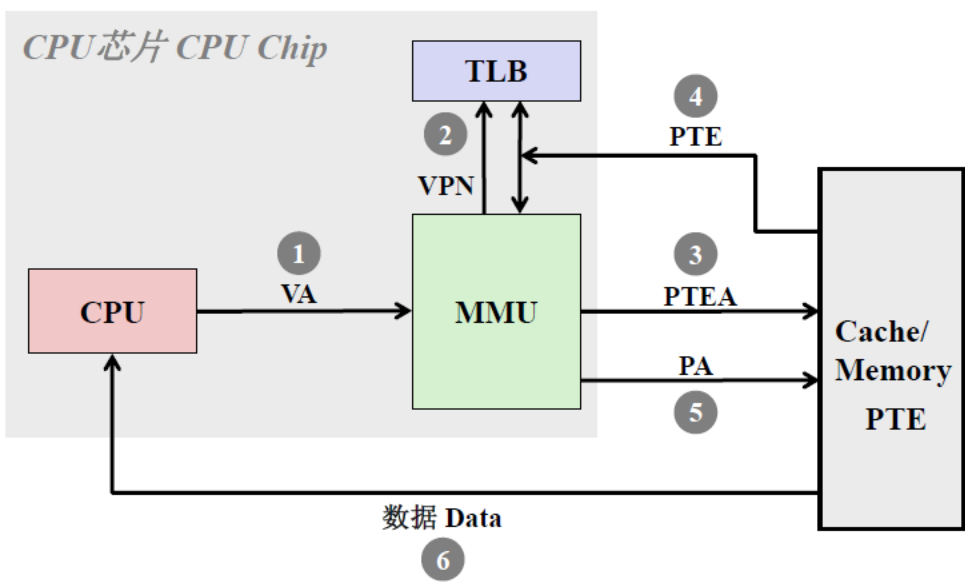
116. 由 VPN 查表(表需要在 Cache 中, 若不在仍需要换入换出), 若有效位=1, 页命中, 得到 PPN, VPO 就是 PP0, 拼接 PPN+VPN 即可



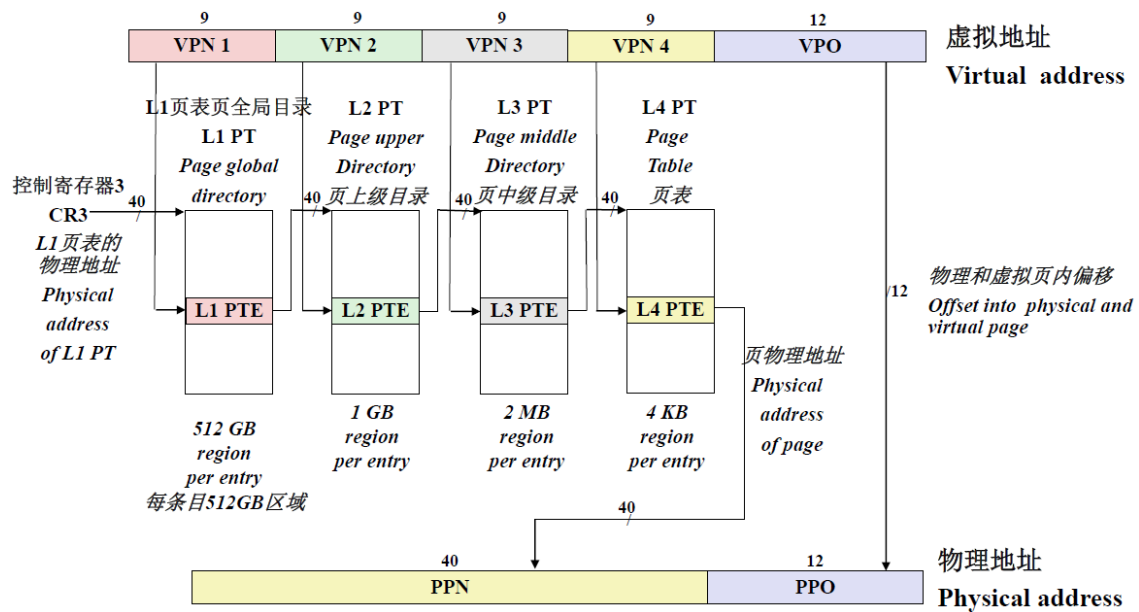
117. 若有效位=0, 触发缺页中断, 从 Cache 中换出页并更新页表, 重新执行中断指令



118. 翻译后备缓冲区 TLB 存储在主存中, 相当于 L1 cache 中的 PTE 的 cache



119. 多级页表: 将 VPN 的位数从高到低分段解释为一级、二级……页表



120. Intel Core i7

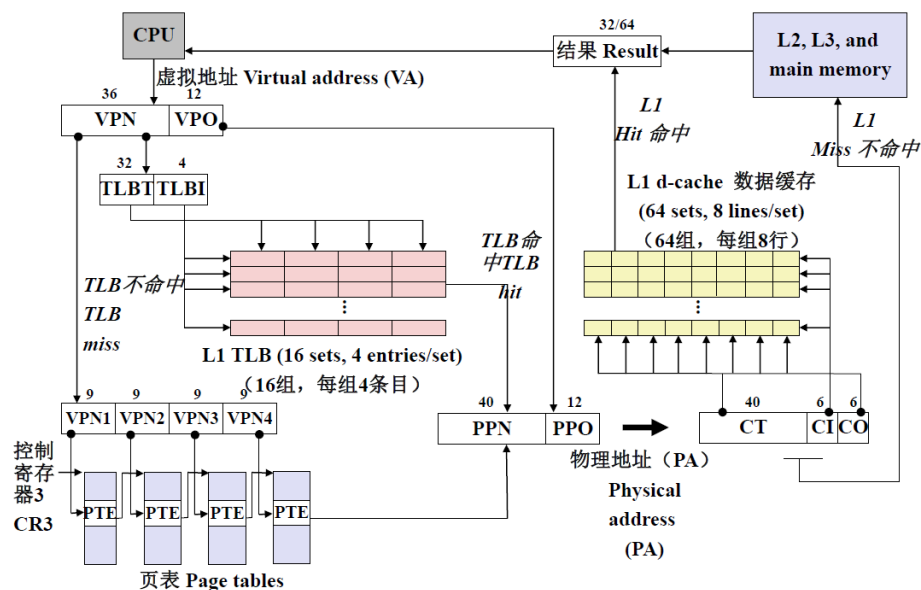
VA: 48 [VPN: 36 (TLBT: 32 + TLBI: 4) + VP0: 12]

PA: 52 [PPN: 40 (CT: 40) + PP0: 12 (CI: 6 + C0: 6)]

将 VPN 分成 4 级页表 (9 位划分为一级)

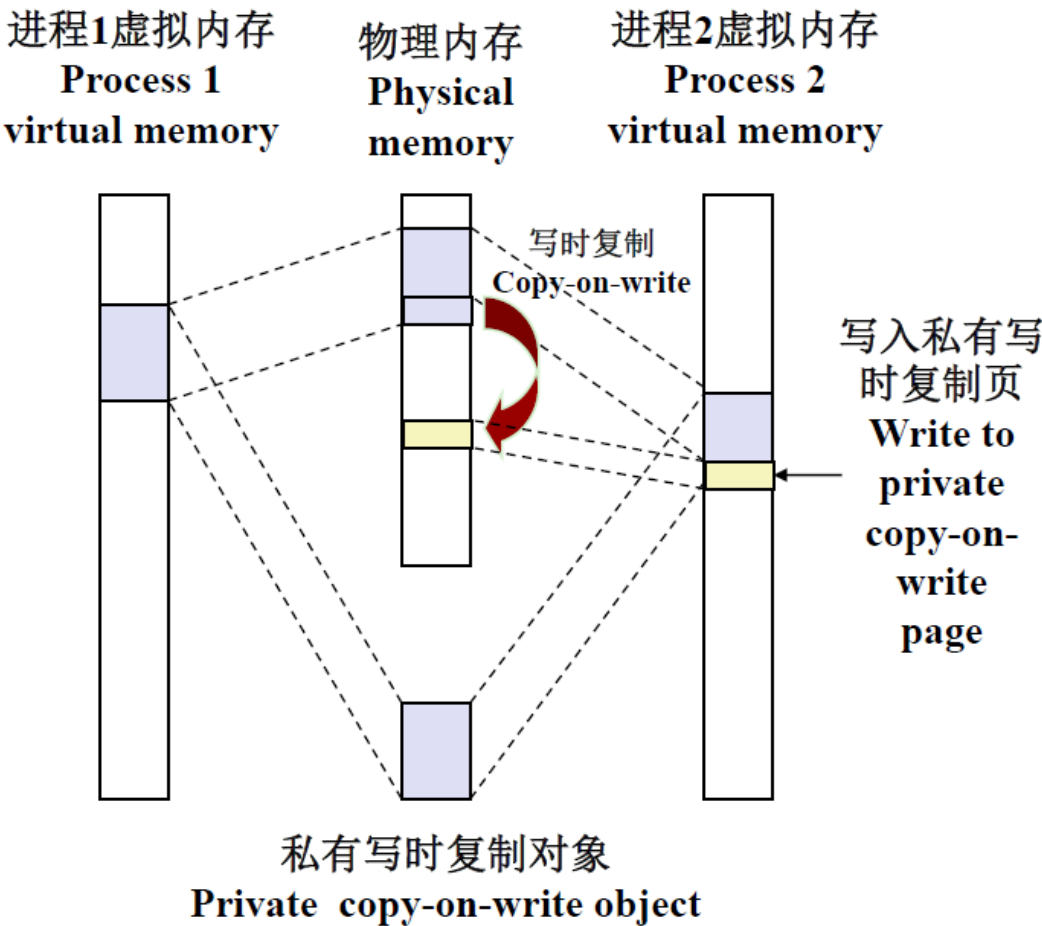
TLB: 16 组 $\times$ 4 个

Cache: 64 组 $\times$ 8 个



通过使 PPN 和 CT 的位数相等，使得在就行地址的虚实翻译的同时就从 Cache 中取数据

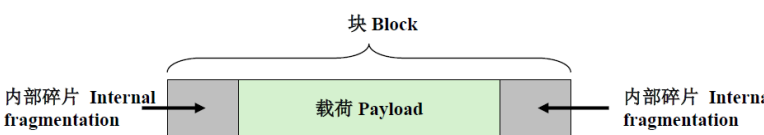
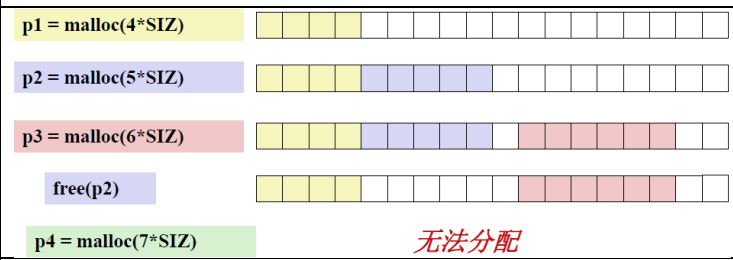
121. 不同进程享有不同虚拟空间，MMU 可能将不同虚拟空间映射到同一物理空间中
假设两个进程不会修改数据，则会将其映射到同一物理空间（存储的是只读的数据）；若进程会修改物理空间中的数据，则会为两个进程分配独立空间



122. 调用 malloc 会申请虚拟内存，在 heap 区域动态分配块，heap 区域不是已分配就是空闲，已分配的块一般无法移动
123. 显式分配器：负责分配和释放；隐式分配器：只分配不释放
124. malloc 上返回的内存地址--x86: 8 字节对齐；x86-64: 16 字节对齐
- 125.

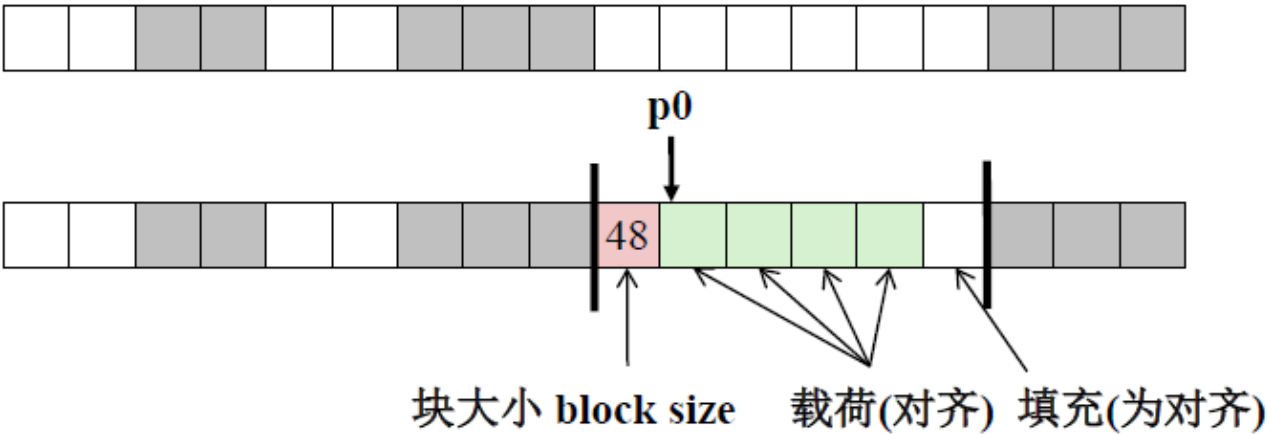
malloc (size)	分配指定字节数的未初始化内存块
calloc (num , size)	分配指定数量和大小的已初始化内存块
realloc (ptr , new-size)	调整已分配内存块的大小

126. 内存碎片

内部碎片	外部碎片
对于给定的块，如果载荷小于块大小就会导致内部碎片	有足够但碎片化的堆内存，但是“没有足够大的连续空闲块”时产生外部碎片
发生在已分配内存块内部	发生在空闲内存区之间
	
① 维护堆数据结构的开销(块头部存储元数据，记录块大小、空闲标记等块信息) ② 对齐填充的部分 ③ 显示分配策略使得分配器故意返回大于请求的内存块	经过一系列分配和释放指令序列后，堆内产生的离散化的空间碎片
大小易于测量	大小难以测量

127. 堆的生命周期由 free 显式释放决定(完全由程序员控制)，栈的生命周期与函数高度绑定(由函数内部逻辑控制)

128. heap 区域中分配块在返回指针的前面会额外分配一个区域用于存储块大小

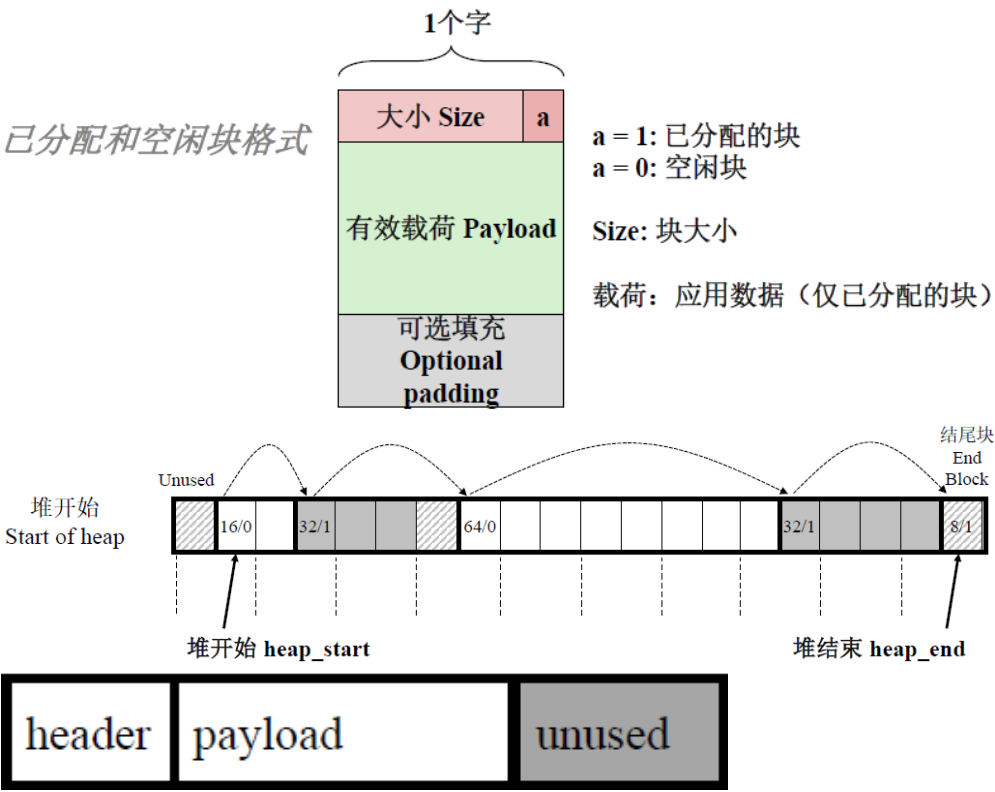


129. 跟踪空闲块的四方法：

① 隐式空闲链表（遍历时会遍历空闲和非空闲的块）

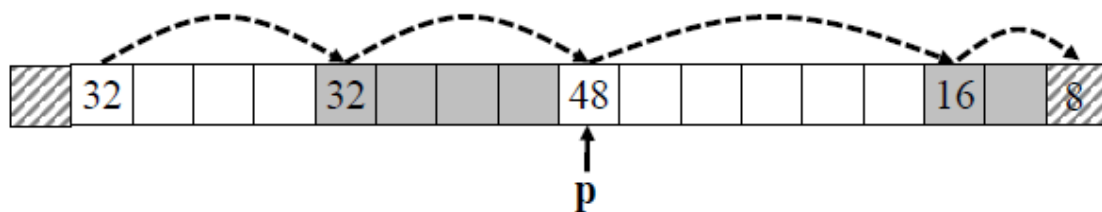
1) 单向隐式链表

因为块是对齐的，则地址低位总是 0，不如作为已分配/空闲的标志位

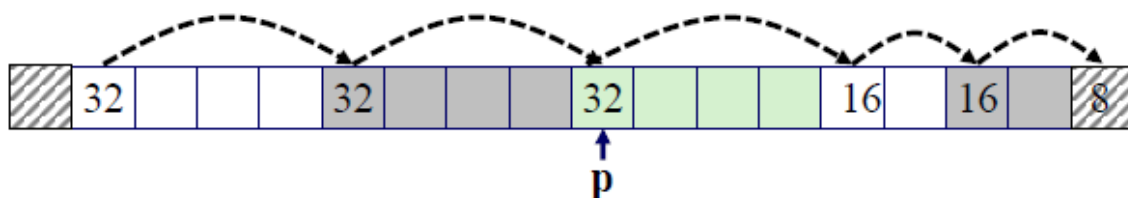


查找空闲块的匹配策略

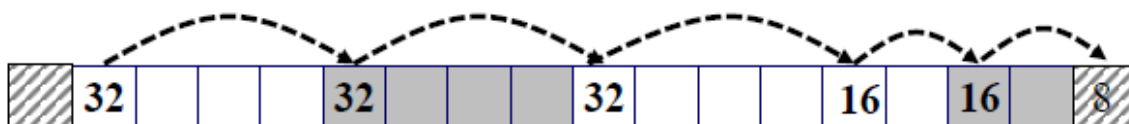
算法	工作原理	开销
首次匹配	从链表头开始扫描，选择首个满足大小要求的空闲块	中
下一次匹配	从上一次扫描结束位置开始，选择首个满足要求的空闲块	大
最佳匹配	遍历整个链表，选择剩余空间最小的合适空闲块	小



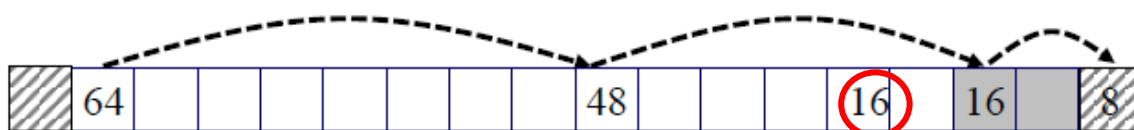
↓空闲块的拆分



↓空闲块的释放(副作用:原本连续的空闲空间不连续了)

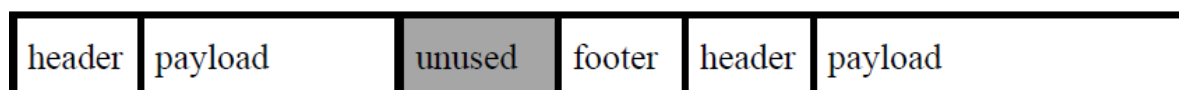
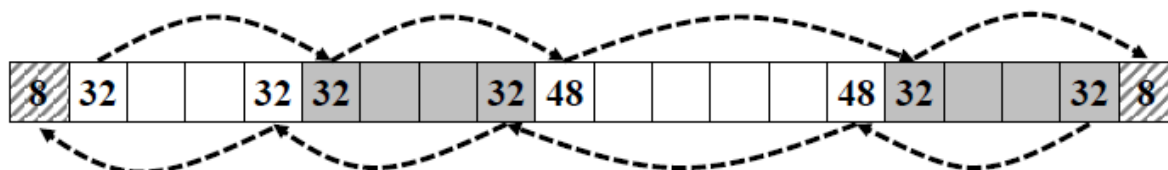
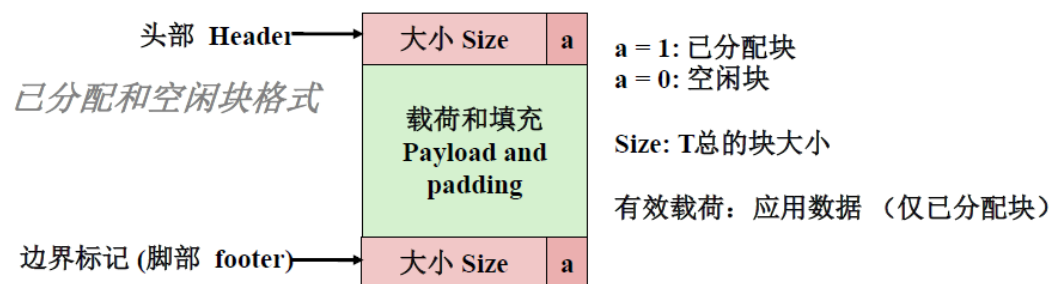


↓空闲块的合并(原来的块头在逻辑上不存在了)



2) 双向隐式链表

边界标记: 在空闲块“底部”(结束)的位置复制块大小/已分配字, 以额外的空间换取反向遍历列表功能



优化：已分配的块不需要脚部的边界标记

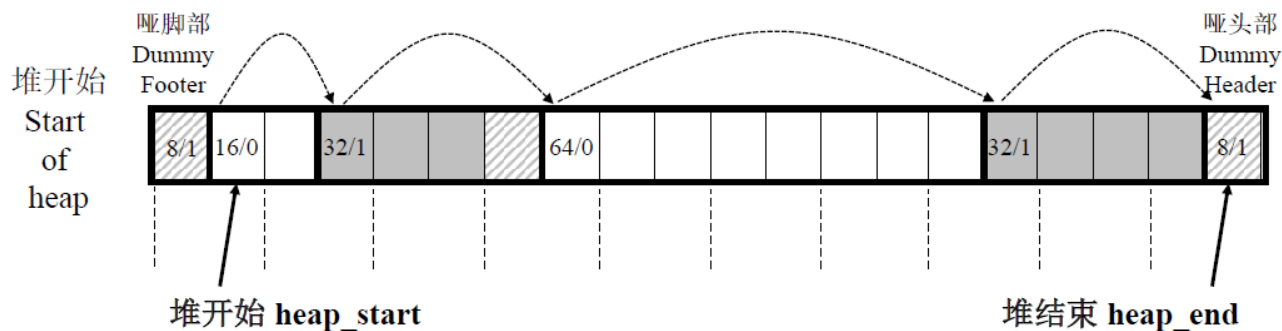
红色标识上一个块的分配状态

蓝色标识下一个块的分配状态

空闲块头脚部的标识一致



2) 堆的结构



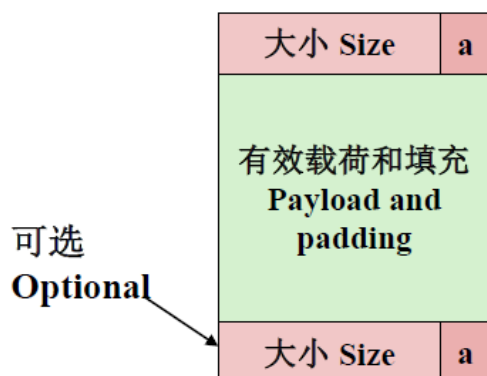
首个头部之前的为哑脚部：标记为已分配，当释放第一个块时，防止意外合并

最后脚部之后的哑头部：在释放最后一块时，防止意外合并

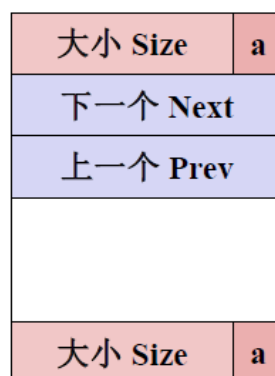
② 显式空闲链表（遍历时仅遍历空闲的块，利用指令跳过非空闲的块）

已分配（和以前一样）

Allocated (as before)

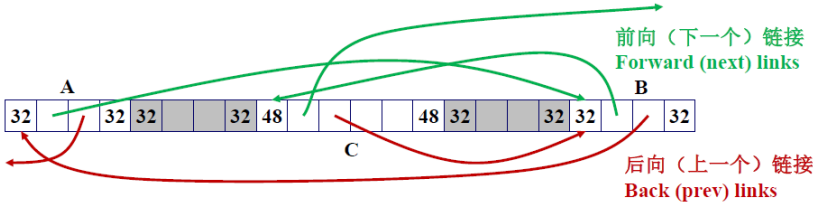


空闲 Free



空闲块新增指向相邻块的指针

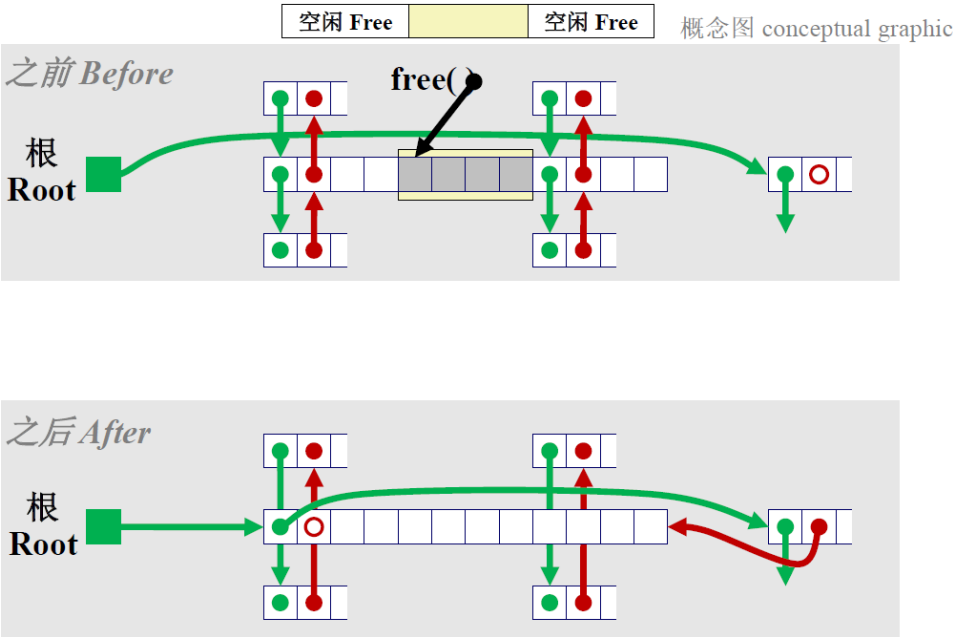
注意：空闲链表的链接不一定按照顺序排布



释放后将空闲块（释放要考虑合并前后的空闲块）插入空闲链表的策略

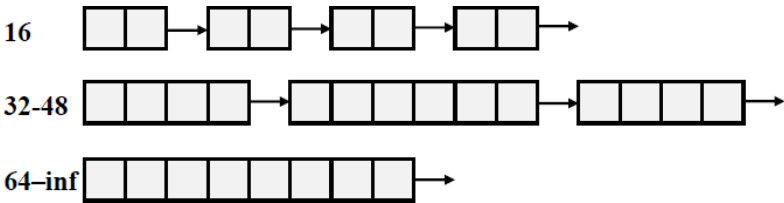
策略	操作	优点	缺点
后进先出	新释放的空闲块 直接插到空闲链表的最开头（相当于头插法）	简单快速，一步到位	更容易产生内存碎片
地址排序	新释放的空闲块 按内存地址大小顺序插入链表	碎片更少，内存利用率更高	操作慢，需遍历链表找到合适插入位置

后进先出考虑以下情况



③ 分离的空闲链表

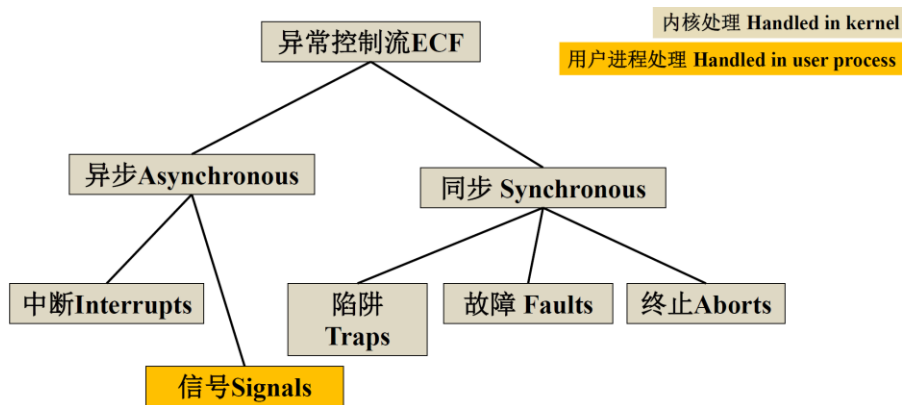
每个不同大小类块有自己的空闲链表



④ 根据大小对块排序：一般使用平衡树（红黑树）

130. strlen(s) 仅会返回字符串 s 中'\0'前的字符的个数，不包括'\0'

131.



分类	触发源	返回方式	示例	可恢复性
中断	时钟、外部 I/O 设备	返回到下一条指令	① 时钟中断 ② 外部 I/O 中断 键盘键入 Ctrl+,、网络有包抵达、从磁盘有数据抵达	可恢复
陷阱	人为的	返回到下一条指令	系统调用、断点、特殊指令	可恢复
故障	程序错误	重新执行当前指令 / 终止执行	缺页异常可恢复、浮点运算异常可恢复、保护异常（非法内存引用）不可恢复	大部分可恢复
终止	硬件故障	终止当前程序执行	非法指令、校验错、机器检查	不可恢复

132. 并行：物理上同时执行多个任务，任务真正同时执行

并行：逻辑上同时处理多个任务，任务快速交替执行

133. 进程的三种状态：运行（正在执行/等待被执行）、停止（挂起）、终止（杀死）

134. 进程终止的三种原因:

- ① 默认动作是终止的信号
- ② 从 main 函数返回
- ③ 调用 exit 函数 (正常退出返回 0，出错返回非 0，进程直接终止不会回到调用位置) 一次调用，不返回

135. fork 函数: 一次调用，两次返回

特性	父进程	子进程
返回值	子进程的 PID (>0)	0
进程 ID (PID)	不变	新分配的唯一 PID
虚拟地址空间	独立不变	父进程的完整拷贝

父子进程的执行顺序不定

只读时父子进程共享同一个内存区域，可写时会拷贝父进程的内存区域给子进程，父子进程内存区域相互独立

136. wait 函数: 挂起当前进程直至任意一个子进程终止，返回终止子进程的 PID
- waitpid 函数: 挂起当前进程直至指定的子进程终止，返回终止子进程的 PID

137. execve 函数: 在当前进程中加载并运行指定的新程序，完全替换原程序的内容 (代码/数据/堆栈)，同时保留进程的 PID、文件描述符和信号上下文，并跳转到新程序的入口点执行，执行成功后永不返回原程序 一次调用，不返回

138. 进程接受到一个信号后①忽略该信号②终止进程③调用用户级信号处理函数对信号进行处理

139. 信号机制的特性是: 同类型信号在挂起时不排队 (新信号丢弃)、进程可选择性阻塞信号接收 (除 SIGKILL/SIGSTOP 外)、且每个挂起信号仅被处理一次。

140. 不能使用信号来计数：内核不记录信号重复次数（仅用单比特位标记是否挂起），连续多次事件可能只触发一次信号处理，导致计数遗漏。

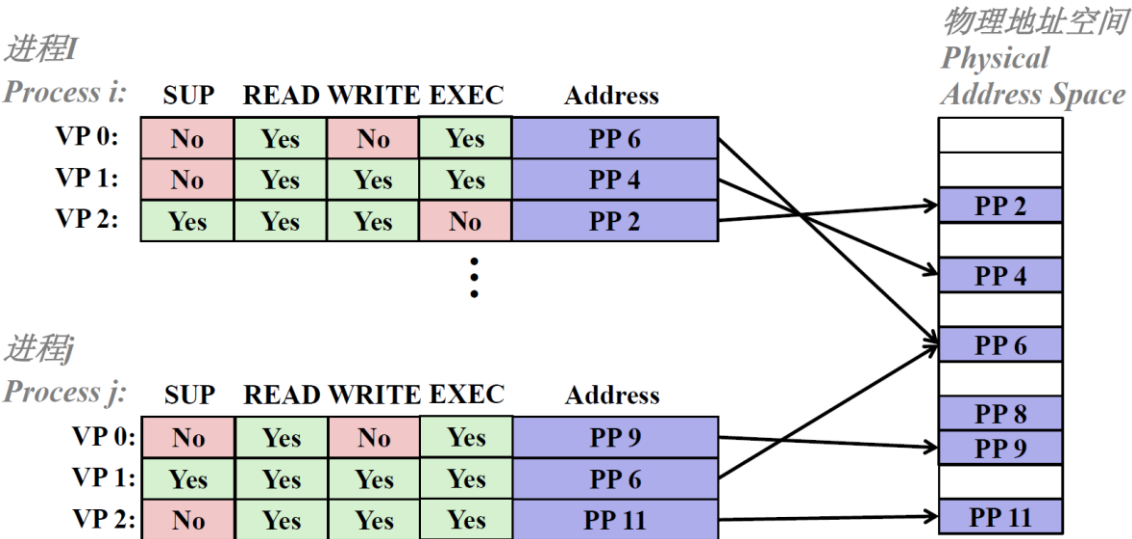
141. `setjmp`：在调用点保存寄存器上下文(PC/栈指针等)到 `jmp_buf` 并首次返回 0，后续 `longjmp(j)` 可直接跳回此点继续执行（返回非零值），实现跨函数栈的任意跳转

142. `longjmp(j, i)`：直接篡改程序计数器（PC）和栈指针，强制跳回 `setjmp` 记录的寄存器上下文位置（绕过函数栈机制），并将该点返回值设为 `i`（若原 `setjmp` 首次返回 0，则此次返回 `i`），实现跨函数的“时空穿越”式跳转

只能跳转到已经调用但是还没有完成的函数

143. 按下 `Ctrl-C` 时：生成 `SIGINT` 信号，立即中断当前执行流，跳转回 `sigsetjmp` 锚点位置（一般设定为主循环起点），重新开始执行主循环→ 实现自重启（进程不终止）

144. `SUP` 的 `Yes/No` 分别标识内核/用户模式，防止用户态应用程序直接访问或修改操作系统内核的敏感数据



145. 可以在命令行后面加上一个“&”将任务放到后台执行

146. 不同映射策略之间的比较

特性	直相联映射	组相联映射	全相联映射
原理	一个内存块固定映射到 Cache 的一个唯一位置	Cache 分为若干组，每组有 N 个行（路），内存块可映射到特定组的任意一行	内存块可映射到 Cache 的任意位置（相当于每组只有 1 行的组相联映射）
查找方式	根据内存地址中间位索引定位 Cache 行，检查标记位是否匹配	1. 用地址中间位选择组 2. 在组内并行比较所有行的标记位	并行比较 Cache 中所有行的标记位
硬件复杂度	⚡ 最低 （无需比较器，直接索引）	⚖️ 中等 （需组内少量并行比较器，路数 N 越大越复杂）	⚠️ 最高 （需大量并行比较器，成本高）
访问速度	⚡ 最快 （单次索引直达）	⚖️ 中等 （组内多路比较略慢）	⚠️ 最慢 （全 Cache 并行比较延迟高）
冲突失效	⚠️ 最高 （多个内存块竞争同一位置易冲突）	⚖️ 中等 （组内多行减少冲突）	✅ 最低 （无位置冲突）
空间利用率	⚠️ 最低 （冲突导致 Cache 行闲置）	⚖️ 中等 （组内灵活替换）	✅ 最高 （完全利用 Cache 空间）
替换策略	无需替换策略（固定位置，新块覆盖旧块）	组内需替换策略（LRU、随机等）	需全局替换策略（LRU、FIFO 等）

